Naija Prime School

Sprint 3 — Students & Parents

Pupils · Parents/Guardians · Linkage · Enrolment

Long-form implementation walk-through

Author: Benjamin Fadina

Branch: sprint/3-students-parents

Built on: Sprint 1 identity foundation + Sprint 2 academic domain

Stack: .NET 10, Blazor Web App (Auto), EF Core 10, SQL Server, Radzen Blazor

Editor: Visual Studio Code with the C# Dev Kit

Repository: https://github.com/benjaminsqlserver/NaijaPrimeSchool

Contents

Contents....2

1. Sprint 3 in context....6

1.1 Where this sits relative to sprint 2....6

1.2 Functional scope delivered....7

1.3 Non-goals deliberately deferred....7

1.4 Scale of the sprint....8

2. Design decisions and trade-offs....9

2.1 Student is its own entity, not an ApplicationUser....9

2.2 No enums — every lookup is a table....9

2.3 Soft delete, again....10

2.4 The StudentParent link is its own entity....10

2.5 Enrolment is per-class, not per-session....10

2.6 Composite unique indexes....11

2.7 Foreign-key delete behaviour....11

2.8 Service per feature area....11

2.9 DateOnly for biographical dates....12

2.10 Inline forms with tabs for related data....12

3. The Domain layer in full....13

3.1 Folder layout and namespacing....13

3.2 Lookup entities....13

3.2.1 Relationship.cs....13

3.2.2 EnrolmentStatus.cs....14

3.2.3 BloodGroup.cs....14

3.2.4 MaritalStatus.cs....14

3.3 Core entities....15

3.3.1 Student.cs....15
3.3.2 Parent.cs....16
3.3.3 StudentParent.cs....17
3.3.4 Enrolment.cs....17
3.4 The one change to sprint 2 — SchoolClass....18
3.5 Relationships at a glance....18
4. Application layer — DTOs and contracts....20
4.1 DTO design rules....20
4.2 StudentDtos.cs....20
4.3 ParentDtos.cs....23
4.4 StudentParentDtos.cs....25
4.5 EnrolmentDtos.cs....26
4.6 IStudentService.cs....27
4.7 IParentService.cs....28
4.8 IEnrolmentService.cs....28
4.9 ILookupService extension....29
5. Infrastructure — DbContext changes....30
5.1 New DbSets....30
5.2 ConfigureFamily....30
5.3 The lookup helper, reused....33
5.4 SaveChanges still does all the work....34
6. Infrastructure — service implementations....35
6.1 StudentService.cs....35
6.2 ParentService.cs....42
6.3 EnrolmentService.cs....46
6.4 LookupService — the seven new methods....50

6.5 DI registration....54
7. The EF Core migration....55
7.1 Indexes created....64
7.2 The schema warning....64
8. Seeding the family lookups....65
9. The Razor pages....69
9.1 Page roster....69
9.2 Students.razor — the directory....69
9.3 CreateStudent.razor — the create form....74
9.4 EditStudent.razor — the heart of the sprint....78
9.5 Parents.razor and CreateParent.razor....93
9.6 EditParent.razor — the parent detail view....100
9.7 Enrolments.razor — the overview....106
10. Navigation, imports, and authorization....111
10.1 NavMenu — a new Family panel....111
10.2 _Imports.razor....111
10.3 Authorization at the page level....111
11. Lifecycle of a family-domain row....112
11.1 Admin opens /students/new....112
11.2 Admin fills the form and clicks Save....112
11.3 Admin links Adaeze's mother....112
11.4 Three years later, Adaeze withdraws....112
11.5 The pupil leaves the school....113
12. Smoke-test walkthrough....114
12.1 Build, migrate, run....114
12.2 Verify navigation....114

12.3 Create a parent....114
12.4 Create a pupil and enrol them....114
12.5 Link the parent....114
12.6 Verify the directory views....114
12.7 Verify the error paths....115
12.8 Verify soft-delete and audit....115
13. Troubleshooting and gotchas....116
13.1 'A student with admission number X already exists'....116
13.2 'Student already has an active enrolment in this session'....116
13.3 'A given parent is already linked to the student'....116
13.4 The parent dropdown is empty....116
13.5 The classes dropdown is empty when creating a student....116
13.6 The 'Cannot delete' message persists after I cleared dependents....116
13.7 The Identity / soft-delete migration warning....116
14. Forward-compatibility, today....117
14.1 What might need a small refactor later....117
15. Appendix — files added or changed in sprint 3....118

1. Sprint 3 in context

Sprint 3 plugs students and their parents into the academic structure laid down in sprint 2. The work in this sprint is the first time the system has anyone in it who is not a member of staff. Pupils get an admission record, a profile, a medical note where it matters, and an enrolment in a class arm; parents and guardians get their own directory; the two sides of the family are stitched together with a many-to-many linkage table that captures relationship, primary contact, and pickup authorisation.

Once this sprint ships, every other module the school is going to need has somewhere to attach to. Attendance hangs off (Student × Class × Date). Result sheets hang off (Student × Subject × Term). Fee invoices hang off (Student × Term × Class). Parent portals hang off the StudentParent linkage. None of those features could land before sprint 3, because there were no students in the system to attach them to.

This document is a long-form implementation guide. It is written so that an engineer who has read the sprint 2 guide and has the codebase checked out can recreate every change in this sprint without referring to the diff. The document is organised in roughly the order I built the code in: design decisions first, Domain entities next, Application contracts after that, Infrastructure (DbContext, services, seeder, migration) in the middle, and finally the Razor UI and navigation. There is a smoke-test chapter near the end that walks through the happy-path you can use to confirm a fresh checkout works.

1.1 Where this sits relative to sprint 2

Sprint 1 delivered authentication, role-based authorization, the BaseEntity pattern, the SaveChanges audit and soft-delete rewriter, the OperationResult shape, and the Radzen-themed admin shell. Sprint 2 layered the academic skeleton on top: Sessions, Terms, Class arms, Subjects, Periods, and the weekly timetable. Both of those sets of foundations are reused unchanged here. In particular:

- BaseEntity — every new entity in sprint 3 inherits from it and picks up Guid Id, IAuditable, and ISoftDelete with no boilerplate.
- ApplicationDbContext.SaveChanges — the override stamps CreatedOn/By and ModifiedOn/By and rewrites Delete to IsDeleted = true. Every family-domain write therefore inherits auditing and soft delete with zero changes to the override.
- Global query filters — every new entity declares HasQueryFilter(x => !x.IsDeleted), so deleted rows vanish from ordinary queries without service code having to remember to filter.
- ICurrentUser / CurrentUserAccessor — the family services do not consume ICurrentUser directly, but the DbContext does, and that is what stamps audit columns when they save.
- OperationResult / OperationResult<T> — every family service uses this for predictable success/failure responses.
- ILookupService — already had eight methods. Sprint 3 adds seven more (relationships, enrolment statuses, blood groups, marital statuses, classes-for-session, students search, parents search) without rewriting the existing ones.
- SchoolClass — sprint 2's class arm picks up one new collection navigation (Enrolments) so the schema can join in the other direction without breaking anything. Existing code that ignored the new property continues to compile and behave identically.

- Radzen Blazor + the green/gold app.css — the family pages adopt the same .nps-page-header / .nps-card / .nps-form-grid primitives so they read as part of the same product, not a bolt-on.

1.2 Functional scope delivered

Concretely, after this sprint a SuperAdmin or HeadTeacher signing in to the application can:

1. Add a new pupil with their identity, demographic profile, blood group, allergies, and medical notes; optionally enrol them into a class straight from the create form.
2. Browse the student directory with full-text search, filter by session and class, and toggle active/inactive pupils.
3. Open a pupil's profile to edit the demographic and medical fields, or to manage their parent linkages and enrolment history.
4. Add a new parent or guardian to the directory; capture title, phone, alternate phone, email, occupation, employer, and marital status.
5. Browse the parents directory with full-text search.
6. From a pupil's profile, link an existing parent to them with a Relationship row (Father, Mother, Uncle, Guardian, Other, …), mark that parent as the primary contact, and toggle whether they are authorised to collect the child.
7. From a parent's profile, see every pupil they are linked to and click straight through to the pupil.
8. Enrol a pupil into a class for a given session, then later withdraw them or move them by enrolling into a different class.
9. Browse all enrolments across the school with session, class, and status filters; withdraw or delete enrolments inline.
10. Soft-delete any of the above, with friendly errors if the row is still referenced elsewhere — you cannot delete a student who has an open enrolment, and you cannot delete a parent who is still linked to a pupil.

Teachers, Bursars, Storekeepers, Parents and Students do not see the Family navigation panel; their menu items remain placeholders reserved for later sprints (parent portal, attendance capture, report cards). Adding a new pupil does not yet create an ApplicationUser for them — there is a deliberate UserId hook on the Student entity that the portal sprint will populate.

1.3 Non-goals deliberately deferred

It is just as important to be explicit about what sprint 3 does NOT do, because every one of these has been weighed and consciously deferred. Trying to land them now would have pushed scope past what is comfortably reviewable in one branch:

- Self-service parent and student logins. The Parent and Student roles are seeded; a UserId column is reserved on each entity. Wiring those up is a separate, opinionated piece of work that belongs to the portal sprint.
- Bulk import (CSV/Excel). Schools generally have a spreadsheet of all current pupils and would like to upload it. The schema will support this without changes; it is a feature-add, not a redesign.

- Photographs. There is a PhotoUrl column on Student but no upload flow yet — schools using the system today can host photos in the wwwroot folder or a CDN and paste a URL. A real upload pipeline (probably backed by a /uploads volume or Azure Blob Storage) will land alongside report cards.
- Sibling discovery. We can find pupils who share a parent through the StudentParent linkage, but there is no dedicated 'siblings' panel on the pupil profile yet. The query is one join; the UI is the lift.
- Attendance. Attendance is sprint 4; the (Student × Class × Date) key it will use is now expressible.
- Promotion / end-of-session bulk roll-over. Moving every pupil from Primary 1A to Primary 2A at the start of a new session is a useful workflow, but it is a one-shot operation against the schema we now have, not a change to the schema.

1.4 Scale of the sprint

By the numbers, this sprint adds:

- 8 new domain entities under src/NaijaPrimeSchool.Domain/Family/.
- 1 new collection navigation on SchoolClass (Enrolments).
- 4 DTO files under src/NaijaPrimeSchool.Application/Family/Dtos/.
- 3 new service contracts under src/NaijaPrimeSchool.Application/Family/.
- 3 new service implementations under src/NaijaPrimeSchool.Infrastructure/Services/.
- 7 new methods on ILookupService (and the matching LookupService).
- 1 EF Core migration introducing 8 new tables and 16 indexes.
- 1 DatabaseInitializer extension seeding Relationships, EnrolmentStatuses, BloodGroups, and MaritalStatuses.
- 6 Razor pages under src/NaijaPrimeSchool.Web/Components/Pages/Family/.
- 1 navigation menu addition to surface the new pages.

Everything compiles with zero warnings on .NET 10 (the team-wide warning bar). The code follows the patterns already accepted in sprints 1 and 2, so the diff is low-friction to review.

2. Design decisions and trade-offs

Before any code was written I made a small number of architectural calls that shaped everything that followed. Re-reading these in isolation makes the code easier to navigate later, and it gives future maintainers permission to revisit choices when the trade-offs change.

2.1 Student is its own entity, not an ApplicationUser

The most consequential decision in this sprint was modelling Student as a first-class entity that is independent of ApplicationUser, rather than making Student a profile attached to an ApplicationUser carrying the Student role. The trade-off is real, so it is worth being explicit:

- Schools admit pupils long before those pupils log in — and many primary-school pupils never will. Forcing a Student to have an Identity row would mean creating throw-away credentials at admission, and asking the bursar's clerk to think about passwords and email addresses while they are filling in a guarantor's telephone number.
- ApplicationUser is shaped for staff: it has UserName, EmailConfirmed, LockoutEnd, PasswordHash, and a host of other Identity columns that are noise for a five-year-old. Subclassing or extending ApplicationUser further would have made the Identity table a junk drawer.
- The Student → ApplicationUser link is still expressible: there is a nullable UserId on Student. The portal sprint will populate it for older pupils who actually need to sign in. Until then, the column is null and the UI does not surface it.
- Several denormalised columns (FirstName, LastName, GenderId, DateOfBirth) live on Student rather than relying on a join. This is intentional: the student record is the source of truth for their identity in the school, even if they later get an ApplicationUser and that login carries different display names.

Parent follows the same shape for the same reasons: one parent in ten gets an actual portal login, the other nine in ten just want phone and email captured so the front office can call them.

2.2 No enums — every lookup is a table

Sprint 1 set the rule that domain concepts which would normally be C# enums are stored as tables instead. Sprint 2 honoured it. Sprint 3 honours it without exception. Relationship, EnrolmentStatus, BloodGroup, and MaritalStatus are all proper entities that derive from BaseEntity and live in the database. The reasons remain the same as the earlier sprints:

- Translatability. A future Hausa or Yoruba translation only needs to edit Name in a row, not recompile a deployment.
- Extensibility by data. If a school wants to add a 'Foster Parent' or 'Step-Grandparent' relationship, the Relationships table accepts a new row; no code change.
- Reportability. Joins to lookup tables surface readable labels in every reporting tool; magic enum integers do not.
- Display order. Each lookup carries a DisplayOrder column which the UI uses to sort dropdowns. Enums would have to lean on the integer value or impose an attribute-based ordering.

BloodGroup is a particularly interesting case here. The set of blood groups is genuinely closed (A+ A- B+ B- AB+ AB- O+ O-, plus Unknown), which is the textbook argument for an enum. I still modelled it as a table

— partly for consistency with the rest of the codebase, partly so that the seed file is the single place a developer reads to discover what the value set is, and partly because '+' and '-' would have made for awkward C# enum members.

2.3 Soft delete, again

Every entity in sprint 3 implements ISoftDelete via BaseEntity. The pattern matches sprint 2 exactly: SaveChanges intercepts EntityState.Deleted, flips IsDeleted to true, and stamps DeletedOn/By. Global query filters then hide the row from every subsequent query that does not explicitly call IgnoreQueryFilters.

What is genuinely new in sprint 3 is the layered pre-delete guards. Three of the family entities refuse a delete that would leave the schema in an awkward state:

- StudentService.SoftDeleteAsync refuses to delete a pupil who still has an open enrolment (WithdrawnOn IS NULL). Withdrawing first forces an audit trail, then the deletion is allowed.
- ParentService.SoftDeleteAsync refuses to delete a parent who is still linked to one or more pupils.
- EnrolmentService allows delete unconditionally — the StudentParent and Student rows that point at it survive because of cascade-on-soft-delete semantics, but historically deleted enrolment rows are rare enough that we do not bother gating them.

An alternative would have been to cascade-soft-delete dependents, so deleting a student also deletes every enrolment and parent link. I chose the explicit refusal because cascade soft delete is hard to undo accurately. Refusing the delete keeps the audit trail readable.

2.4 The StudentParent link is its own entity

EF Core 10 handles many-to-many through a join entity, and we could have used the implicit pattern (just navigation collections and let EF magic up the join table). I made StudentParent an explicit entity for two reasons:

- It carries data in its own right — RelationshipId, IsPrimaryContact, CanPickUp, Notes. Implicit join tables cannot do that without ceremony.
- Soft delete needs an explicit type. The implicit join entity does not implement ISoftDelete; making it explicit lets us reuse the BaseEntity pattern unchanged.

The unique index (StudentId, ParentId) ensures a given parent appears at most once per pupil. The service layer also pre-checks this in LinkParentAsync so the user gets a friendly OperationResult.Failure rather than a database exception.

2.5 Enrolment is per-class, not per-session

An enrolment row points at a SchoolClass. Because every SchoolClass already belongs to a Session (sprint 2), the session is implicit — there is no SessionId column on Enrolment. This matters because:

- Reading 'who is in Primary 1A in 2025/2026' is a simple WHERE SchoolClassId = …, no extra join.
- Moving a pupil from Primary 1A to Primary 1B in the middle of a term is just two enrolment rows: one withdrawn, one new.

- There can never be a contradiction between Enrolment.SessionId and SchoolClass.SessionId, because the former does not exist.

The cost is that 'show me a pupil's history across sessions' joins through SchoolClass to Session. EF Core handles that elegantly in the projection (see EnrolmentService.Project), and the cost is a single extra index lookup on a tiny Sessions table.

The service layer enforces 'one open enrolment per session per pupil' explicitly: CreateAsync rejects an enrolment if the pupil already has an enrolment with WithdrawnOn IS NULL in the same session. That keeps a pupil from being in Primary 1A and Primary 1B simultaneously.

2.6 Composite unique indexes

Two composite unique indexes do most of the structural integrity heavy lifting in sprint 3:

- (StudentId, ParentId) on StudentParent — a parent cannot be linked twice to the same pupil.
- (StudentId, SchoolClassId) on Enrolment — a pupil cannot be enrolled twice in the same class arm. Re-enrolment after a withdrawal would have to be a new row in a new class.

Plus a filtered unique index on Student.UserId (and Parent.UserId) with [UserId] IS NOT NULL — sprint 8 will use this to ensure each pupil/parent maps to at most one Identity user, while still allowing many rows to have a null UserId today.

2.7 Foreign-key delete behaviour

Family relationships use a deliberately mixed delete behaviour:

- StudentParent.StudentId / ParentId → Cascade. If a student row ever gets hard-deleted (e.g. via a script that bypasses the soft-delete pattern), their links go with them. We cannot reach this state from the UI but the schema is honest about it.
- Enrolment.StudentId → Cascade for the same reason.
- Enrolment.SchoolClassId → Restrict. A class arm cannot be hard-deleted while it still has enrolments. This matches the soft-delete UI flow that already refuses to delete a class with timetable entries.
- Lookup foreign keys (RelationshipId, EnrolmentStatusId, BloodGroupId, MaritalStatusId) → SetNull where the field is optional, Restrict where it is required. You cannot accidentally lose 'Father' from the Relationships table while there are linkages pointing at it.
- UserId on Student and Parent → SetNull. Removing an Identity user should not delete the underlying student or parent.

2.8 Service per feature area

Three new services split the work along feature lines: IStudentService, IParentService, and IEnrolmentService. The StudentService is the heaviest, because it owns the StudentParent linkage operations as well — the linkage is conceptually 'a thing the student has', and a separate IStudentParentService felt like over-engineering for a half-dozen methods.

Likewise, ILookupService grew by seven methods rather than spawning a separate IFamilyLookupService. The lookup interface is the UI-facing dropdown source for the entire app; fragmenting it would have just meant Razor pages injecting two services where they used to inject one.

2.9 DateOnly for biographical dates

Student.DateOfBirth, Student.AdmissionDate, Enrolment.EnrolledOn, and Enrolment.WithdrawnOn are all DateOnly. The reasoning is the same as in sprint 2: SQL Server's date column is the right storage type, and DateOnly avoids the time-zone landmines that DateTime would introduce. The Razor pages still bind Radzen DatePickers to DateTime? in the form model and convert at the boundary using DateOnly.FromDateTime — Radzen has not yet adopted DateOnly natively.

2.10 Inline forms with tabs for related data

EditStudent.razor and EditParent.razor introduce a pattern that did not exist in sprint 2: a Radzen tab strip with the profile form on the first tab, related data on subsequent tabs. The Students profile has Profile / Parents / Enrolment history; the Parent profile has Profile / Linked pupils. The reasoning:

- Editing demographics and managing parent links are conceptually different tasks — separating them prevents a 'wall of fields' form.
- Each tab body is itself an inline form (the parent-link tab shows a small RadzenCard form when you click 'Link a parent'), so the sprint 2 inline-form pattern is preserved on the inside.
- Tabs avoid Radzen dialogs, which the sprint 2 design notes rejected for accessibility and testability.

3. The Domain layer in full

Every family-domain entity lives in a single new folder, src/NaijaPrimeSchool.Domain/Family/. There are no abstract base classes other than BaseEntity, and the entities are deliberately anaemic — no domain methods, no validation logic. Validation lives in the Application DTOs (DataAnnotations) and in the Infrastructure services (cross-aggregate checks). The Domain layer is a typed vocabulary; it is not the place for behaviour in this codebase.

3.1 Folder layout and namespacing

```
src/NaijaPrimeSchool.Domain/
├── Family/                      ← (new in sprint 3)
│   ├── Relationship.cs          ← lookup
│   ├── EnrolmentStatus.cs       ← lookup
│   ├── BloodGroup.cs            ← lookup
│   ├── MaritalStatus.cs         ← lookup
│   ├── Student.cs               ← pupil
│   ├── Parent.cs                ← parent / guardian
│   ├── StudentParent.cs         ← linkage with relationship
│   └── Enrolment.cs             ← (student × class) over time
├── Academics/                   ← from sprint 2 (SchoolClass updated)
├── Common/                      ← from sprint 1
└── Identity/                    ← from sprint 1
```

Namespace is NaijaPrimeSchool.Domain.Family. Lookup entities live alongside their owners in Family/ rather than in a separate Lookups folder, mirroring the sprint 2 layout where TermType and ClassLevel sit beside Term and SchoolClass.

3.2 Lookup entities

3.2.1 Relationship.cs

Relationship is the lookup of how a parent stands to the child: Father, Mother, Stepfather, Stepmother, Grandfather, Grandmother, Uncle, Aunt, Guardian, Other. The seed list is in DatabaseInitializer; adding Foster Parent or Sibling is a row insertion, not a code change.

Listing — src/NaijaPrimeSchool.Domain/Family/Relationship.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Family;

public class Relationship : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    public ICollection<StudentParent> StudentLinks { get; set; } = [];
}
```

3.2.2 EnrolmentStatus.cs

EnrolmentStatus is the lookup of what state an enrolment is in: Active, Suspended, Withdrawn, Transferred, Graduated. The service layer treats 'Withdrawn' specially — the WithdrawAsync convenience method looks up that row by name and uses it. If a school renames the row, WithdrawAsync still works because WithdrawnOn is the load-bearing column, not the status name.

Listing — src/NaijaPrimeSchool.Domain/Family/EnrolmentStatus.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Family;

public class EnrolmentStatus : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    public ICollection<Enrolment> Enrolments { get; set; } = [];
}
```

3.2.3 BloodGroup.cs

BloodGroup carries the eight blood types plus an explicit 'Unknown' so that captured-but-not-recorded is distinguishable from never-asked. The Name column is 10 characters wide, which is plenty for 'AB+' and 'Unknown'.

Listing — src/NaijaPrimeSchool.Domain/Family/BloodGroup.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Family;

public class BloodGroup : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    public ICollection<Student> Students { get; set; } = [];
}
```

3.2.4 MaritalStatus.cs

MaritalStatus covers Single, Married, Divorced, Widowed, Separated. It feeds the Parent profile — which rarely matters to the school but matters for the records that the school regulator wants once a year.

Listing — src/NaijaPrimeSchool.Domain/Family/MaritalStatus.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Family;

public class MaritalStatus : BaseEntity
{
```

```
    public string Name { get; set; } = string.Empty;
    public int DisplayOrder { get; set; }

    public ICollection<Parent> Parents { get; set; } = [];
}
```

3.3 Core entities

3.3.1 Student.cs

Student is the pupil. AdmissionNumber is the unique admin handle (NPS/2026/0001). FirstName / LastName / MiddleName are denormalised so that the student record is the source of truth for their identity in the school. The optional UserId hooks the pupil to an ApplicationUser if and when a portal login is needed; the unique filtered index ensures one ApplicationUser maps to at most one pupil.

Listing — src/NaijaPrimeSchool.Domain/Family/Student.cs

```
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Identity;

namespace NaijaPrimeSchool.Domain.Family;

public class Student : BaseEntity
{
    public string AdmissionNumber { get; set; } = string.Empty;
    public DateOnly AdmissionDate { get; set; }

    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string? MiddleName { get; set; }

    public DateOnly DateOfBirth { get; set; }

    public Guid? GenderId { get; set; }
    public Gender? Gender { get; set; }

    public Guid? BloodGroupId { get; set; }
    public BloodGroup? BloodGroup { get; set; }

    public string? StateOfOrigin { get; set; }
    public string? ResidentialAddress { get; set; }
    public string? PhotoUrl { get; set; }
    public string? Allergies { get; set; }
    public string? MedicalNotes { get; set; }

    public Guid? UserId { get; set; }
    public ApplicationUser? User { get; set; }

    public bool IsActive { get; set; } = true;

    public ICollection<Enrolment> Enrolments { get; set; } = [];
    public ICollection<StudentParent> ParentLinks { get; set; } = [];
```

```
    public string FullName =>
        string.IsNullOrWhiteSpace(MiddleName)
            ? $"{FirstName} {LastName}".Trim()
            : $"{FirstName} {MiddleName} {LastName}".Trim();
}
```

FullName is a computed property that the DbContext explicitly ignores in the model configuration (see chapter 5). Allergies and MedicalNotes are deliberately free-text — schools have a lot of variation in how they record them, and an enum or table here would be brittle.

3.3.2 Parent.cs

Parent mirrors Student in shape but is shaped for adults: Title, MaritalStatus, PrimaryPhone, AlternatePhone, Email, Occupation, Employer. There is no admission number — a parent is not 'admitted' to the school. There is no DateOfBirth either, because no school I have built for has ever asked the parent's DOB and a few have politely declined to ask.

Listing — src/NaijaPrimeSchool.Domain/Family/Parent.cs

```
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Identity;

namespace NaijaPrimeSchool.Domain.Family;

public class Parent : BaseEntity
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string? MiddleName { get; set; }

    public Guid? TitleId { get; set; }
    public Title? Title { get; set; }

    public Guid? GenderId { get; set; }
    public Gender? Gender { get; set; }

    public Guid? MaritalStatusId { get; set; }
    public MaritalStatus? MaritalStatus { get; set; }

    public string? PrimaryPhone { get; set; }
    public string? AlternatePhone { get; set; }
    public string? Email { get; set; }
    public string? ResidentialAddress { get; set; }
    public string? Occupation { get; set; }
    public string? Employer { get; set; }

    public Guid? UserId { get; set; }
    public ApplicationUser? User { get; set; }

    public bool IsActive { get; set; } = true;

    public ICollection<StudentParent> StudentLinks { get; set; } = [];
```

```
    public string FullName =>
        string.IsNullOrWhiteSpace(MiddleName)
            ? $"{FirstName} {LastName}".Trim()
            : $"{FirstName} {MiddleName} {LastName}".Trim();
}
```

3.3.3 StudentParent.cs

StudentParent is the explicit join entity between Student and Parent. RelationshipId tells you what hat the parent wears for this child; IsPrimaryContact picks one parent for school-day phone calls; CanPickUp gates pickup at the gate. Notes is a free-text annotation for special instructions ('only collected by mother on Tuesdays').

Listing — src/NaijaPrimeSchool.Domain/Family/StudentParent.cs

```
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Family;

public class StudentParent : BaseEntity
{
    public Guid StudentId { get; set; }
    public Student? Student { get; set; }

    public Guid ParentId { get; set; }
    public Parent? Parent { get; set; }

    public Guid RelationshipId { get; set; }
    public Relationship? Relationship { get; set; }

    public bool IsPrimaryContact { get; set; }
    public bool CanPickUp { get; set; } = true;
    public string? Notes { get; set; }
}
```

Note that there is no StudentName / ParentName denormalisation. StudentParent reaches across navigations to surface the names in the DTOs; the join entity itself stays narrow.

3.3.4 Enrolment.cs

Enrolment links Student to SchoolClass with a date range and a status. EnrolledOn is required; WithdrawnOn is null while the enrolment is active. Notes is a free-text annotation for the registrar — typical entries are 'transferred from St George's' or 'parents requested mid-year change of class'.

Listing — src/NaijaPrimeSchool.Domain/Family/Enrolment.cs

```
using NaijaPrimeSchool.Domain.Academics;
using NaijaPrimeSchool.Domain.Common;

namespace NaijaPrimeSchool.Domain.Family;

public class Enrolment : BaseEntity
{
```

```
    public Guid StudentId { get; set; }
    public Student? Student { get; set; }

    public Guid SchoolClassId { get; set; }
    public SchoolClass? SchoolClass { get; set; }

    public DateOnly EnrolledOn { get; set; }
    public DateOnly? WithdrawnOn { get; set; }

    public Guid EnrolmentStatusId { get; set; }
    public EnrolmentStatus? EnrolmentStatus { get; set; }

    public string? Notes { get; set; }
}
```

3.4 The one change to sprint 2 — SchoolClass

SchoolClass picks up exactly one new line: a collection navigation called Enrolments. Existing code that ignored this property continues to compile and behave identically; the migration emits a foreign-key index but no schema change to SchoolClasses itself. This is the smallest possible change to an existing domain entity.

Listing — src/NaijaPrimeSchool.Domain/Academics/SchoolClass.cs

```
using NaijaPrimeSchool.Domain.Common;
using NaijaPrimeSchool.Domain.Family;
using NaijaPrimeSchool.Domain.Identity;

namespace NaijaPrimeSchool.Domain.Academics;

public class SchoolClass : BaseEntity
{
    public string Name { get; set; } = string.Empty;
    public string? Description { get; set; }

    public Guid ClassLevelId { get; set; }
    public ClassLevel? ClassLevel { get; set; }

    public Guid SessionId { get; set; }
    public Session? Session { get; set; }

    public Guid? ClassTeacherId { get; set; }
    public ApplicationUser? ClassTeacher { get; set; }

    public ICollection<TimetableEntry> TimetableEntries { get; set; } = [];
    public ICollection<Enrolment> Enrolments { get; set; } = [];
}
```

3.5 Relationships at a glance

An ASCII relationship diagram captures the family-side of the schema and how it joins back to academics:

```
                    ┌──────────────┐                    ┌──────────────┐
```

```
                    │   Student    │                    │    Parent    │
                    │  Admission#  │                    │   Phone(s)   │
                    │  IsActive    │                    │   IsActive   │
                    └──────────────┘                    └──────────────┘
                       │          │                        │
                       │ N      │ N                      │ N
                       │        │                        │
                       │  ┌─────▼──────────┐             │
                       │  │ StudentParent│ N────────────┘
                       │  │ Relationship │
                       │  │ IsPrimary    │
                       │  │ CanPickUp    │
                       │  └────────────────┘
                       │
                       │ 1
                ┌──────▼──────────┐
                │   Enrolment    │ N─── EnrolmentStatus (lookup)
                │   EnrolledOn   │
                │   WithdrawnOn  │
                └────────────────┘
                       │
                       │ N
                ┌──────▼──────────┐
                │  SchoolClass   │  ←── (sprint 2)
                │   Session      │
                └────────────────┘


    Lookups (BaseEntity, soft-deletable):
      Relationship, EnrolmentStatus, BloodGroup, MaritalStatus
    Cross-domain re-uses:
      Student.GenderId       → Identity.Gender   (sprint 1 lookup)
      Student.BloodGroupId   → Family.BloodGroup
      Parent.TitleId         → Identity.Title    (sprint 1 lookup)
      Parent.MaritalStatusId → Family.MaritalStatus
      Student.UserId / Parent.UserId → Identity.ApplicationUser  (nullable)
```

4. Application layer — DTOs and contracts

Application is the boundary between the Razor UI and the persistence layer. It is deliberately thin: just DTOs and service interfaces. There is no domain logic and no EF Core. Sprint 3 adds two new subfolders: Family/Dtos and Family. The DTOs live in the first; the contracts in the second.

4.1 DTO design rules

- Every list view returns a flat read-DTO with denormalised display fields. StudentDto carries CurrentClassName, CurrentSessionName, PrimaryContactName, and PrimaryContactPhone so the list view can render a row without lazy-loading any navigations.
- Create and Update requests are separate types with DataAnnotations validation, mirroring the sprint 2 pattern.
- Filter types (StudentListFilter, ParentListFilter, EnrolmentListFilter) carry the search/sort/page parameters and let the service layer translate into LINQ.
- Linkage DTOs (StudentParentDto, LinkStudentParentRequest, UpdateStudentParentLinkRequest) live in their own file because they are conceptually a separate concept from either the student or parent.

4.2 StudentDtos.cs

Listing — src/NaijaPrimeSchool.Application/Family/Dtos/StudentDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Family.Dtos;

public class StudentDto
{
    public Guid Id { get; set; }
    public string AdmissionNumber { get; set; } = string.Empty;
    public DateOnly AdmissionDate { get; set; }

    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string? MiddleName { get; set; }
    public string FullName { get; set; } = string.Empty;

    public DateOnly DateOfBirth { get; set; }

    public Guid? GenderId { get; set; }
    public string? GenderName { get; set; }

    public Guid? BloodGroupId { get; set; }
    public string? BloodGroupName { get; set; }

    public string? StateOfOrigin { get; set; }
    public string? ResidentialAddress { get; set; }
    public string? PhotoUrl { get; set; }
    public string? Allergies { get; set; }
    public string? MedicalNotes { get; set; }

    public bool IsActive { get; set; }
```

```
    public Guid? CurrentClassId { get; set; }
    public string? CurrentClassName { get; set; }
    public Guid? CurrentSessionId { get; set; }
    public string? CurrentSessionName { get; set; }

    public int ParentCount { get; set; }
    public string? PrimaryContactName { get; set; }
    public string? PrimaryContactPhone { get; set; }
}

public class CreateStudentRequest
{
    [Required, StringLength(30)]
    public string AdmissionNumber { get; set; } = string.Empty;

    [Required] public DateOnly AdmissionDate { get; set; }

    [Required, StringLength(80)]
    public string FirstName { get; set; } = string.Empty;

    [Required, StringLength(80)]
    public string LastName { get; set; } = string.Empty;

    [StringLength(80)]
    public string? MiddleName { get; set; }

    [Required] public DateOnly DateOfBirth { get; set; }

    public Guid? GenderId { get; set; }
    public Guid? BloodGroupId { get; set; }

    [StringLength(80)]
    public string? StateOfOrigin { get; set; }

    [StringLength(300)]
    public string? ResidentialAddress { get; set; }

    [StringLength(500)]
    public string? PhotoUrl { get; set; }

    [StringLength(500)]
    public string? Allergies { get; set; }

    [StringLength(1000)]
    public string? MedicalNotes { get; set; }

    public bool IsActive { get; set; } = true;

    public Guid? InitialClassId { get; set; }
}

public class UpdateStudentRequest
{
    public Guid Id { get; set; }
```

```
    [Required, StringLength(30)]
    public string AdmissionNumber { get; set; } = string.Empty;

    [Required] public DateOnly AdmissionDate { get; set; }

    [Required, StringLength(80)]
    public string FirstName { get; set; } = string.Empty;

    [Required, StringLength(80)]
    public string LastName { get; set; } = string.Empty;

    [StringLength(80)]
    public string? MiddleName { get; set; }

    [Required] public DateOnly DateOfBirth { get; set; }

    public Guid? GenderId { get; set; }
    public Guid? BloodGroupId { get; set; }

    [StringLength(80)]
    public string? StateOfOrigin { get; set; }

    [StringLength(300)]
    public string? ResidentialAddress { get; set; }

    [StringLength(500)]
    public string? PhotoUrl { get; set; }

    [StringLength(500)]
    public string? Allergies { get; set; }

    [StringLength(1000)]
    public string? MedicalNotes { get; set; }
}

public class StudentListFilter
{
    public string? SearchTerm { get; set; }
    public Guid? ClassId { get; set; }
    public Guid? SessionId { get; set; }
    public bool? IsActive { get; set; }
    public int Skip { get; set; }
    public int Take { get; set; } = 20;
    public string? OrderBy { get; set; }
}
```

CurrentClassId, CurrentClassName, CurrentSessionId, CurrentSessionName, ParentCount, PrimaryContactName, and PrimaryContactPhone are populated by the service via a subquery in the projection — see chapter 6. The list view therefore renders the most useful information about a pupil without needing follow-up reads.

4.3 ParentDtos.cs

Listing — src/NaijaPrimeSchool.Application/Family/Dtos/ParentDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Family.Dtos;

public class ParentDto
{
    public Guid Id { get; set; }

    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string? MiddleName { get; set; }
    public string FullName { get; set; } = string.Empty;

    public Guid? TitleId { get; set; }
    public string? TitleName { get; set; }

    public Guid? GenderId { get; set; }
    public string? GenderName { get; set; }

    public Guid? MaritalStatusId { get; set; }
    public string? MaritalStatusName { get; set; }

    public string? PrimaryPhone { get; set; }
    public string? AlternatePhone { get; set; }
    public string? Email { get; set; }
    public string? ResidentialAddress { get; set; }
    public string? Occupation { get; set; }
    public string? Employer { get; set; }

    public bool IsActive { get; set; }

    public int StudentCount { get; set; }
}

public class CreateParentRequest
{
    [Required, StringLength(80)]
    public string FirstName { get; set; } = string.Empty;

    [Required, StringLength(80)]
    public string LastName { get; set; } = string.Empty;

    [StringLength(80)]
    public string? MiddleName { get; set; }

    public Guid? TitleId { get; set; }
    public Guid? GenderId { get; set; }
    public Guid? MaritalStatusId { get; set; }

    [Phone, StringLength(30)]
    public string? PrimaryPhone { get; set; }
```

```
    [Phone, StringLength(30)]
    public string? AlternatePhone { get; set; }

    [EmailAddress, StringLength(256)]
    public string? Email { get; set; }

    [StringLength(300)]
    public string? ResidentialAddress { get; set; }

    [StringLength(120)]
    public string? Occupation { get; set; }

    [StringLength(120)]
    public string? Employer { get; set; }

    public bool IsActive { get; set; } = true;
}

public class UpdateParentRequest
{
    public Guid Id { get; set; }

    [Required, StringLength(80)]
    public string FirstName { get; set; } = string.Empty;

    [Required, StringLength(80)]
    public string LastName { get; set; } = string.Empty;

    [StringLength(80)]
    public string? MiddleName { get; set; }

    public Guid? TitleId { get; set; }
    public Guid? GenderId { get; set; }
    public Guid? MaritalStatusId { get; set; }

    [Phone, StringLength(30)]
    public string? PrimaryPhone { get; set; }

    [Phone, StringLength(30)]
    public string? AlternatePhone { get; set; }

    [EmailAddress, StringLength(256)]
    public string? Email { get; set; }

    [StringLength(300)]
    public string? ResidentialAddress { get; set; }

    [StringLength(120)]
    public string? Occupation { get; set; }

    [StringLength(120)]
    public string? Employer { get; set; }
}
```

```
public class ParentListFilter
{
    public string? SearchTerm { get; set; }
    public bool? IsActive { get; set; }
    public int Skip { get; set; }
    public int Take { get; set; } = 20;
    public string? OrderBy { get; set; }
}
```

StudentCount on ParentDto is the count of linked pupils, computed via .StudentLinks.Count in the projection. It lets the parents grid show, at a glance, which entries in the directory are still tied to active children.

4.4 StudentParentDtos.cs

Listing — src/NaijaPrimeSchool.Application/Family/Dtos/StudentParentDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Family.Dtos;

public class StudentParentDto
{
    public Guid Id { get; set; }

    public Guid StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public string StudentAdmissionNumber { get; set; } = string.Empty;

    public Guid ParentId { get; set; }
    public string ParentName { get; set; } = string.Empty;
    public string? ParentPhone { get; set; }
    public string? ParentEmail { get; set; }

    public Guid RelationshipId { get; set; }
    public string RelationshipName { get; set; } = string.Empty;

    public bool IsPrimaryContact { get; set; }
    public bool CanPickUp { get; set; }
    public string? Notes { get; set; }
}

public class LinkStudentParentRequest
{
    [Required] public Guid StudentId { get; set; }
    [Required] public Guid ParentId { get; set; }
    [Required] public Guid RelationshipId { get; set; }
    public bool IsPrimaryContact { get; set; }
    public bool CanPickUp { get; set; } = true;

    [StringLength(300)]
    public string? Notes { get; set; }
}
```

```
public class UpdateStudentParentLinkRequest
{
    public Guid Id { get; set; }
    [Required] public Guid RelationshipId { get; set; }
    public bool IsPrimaryContact { get; set; }
    public bool CanPickUp { get; set; } = true;

    [StringLength(300)]
    public string? Notes { get; set; }
}
```

There is a deliberate asymmetry between LinkStudentParentRequest (creates a new link) and UpdateStudentParentLinkRequest (edits an existing link). The create form needs StudentId and ParentId; the update form does not, because the link's identity is its Id and the underlying pupil and parent should not change.

4.5 EnrolmentDtos.cs

Listing — src/NaijaPrimeSchool.Application/Family/Dtos/EnrolmentDtos.cs

```
using System.ComponentModel.DataAnnotations;

namespace NaijaPrimeSchool.Application.Family.Dtos;

public class EnrolmentDto
{
    public Guid Id { get; set; }

    public Guid StudentId { get; set; }
    public string StudentName { get; set; } = string.Empty;
    public string StudentAdmissionNumber { get; set; } = string.Empty;

    public Guid SchoolClassId { get; set; }
    public string SchoolClassName { get; set; } = string.Empty;

    public Guid SessionId { get; set; }
    public string SessionName { get; set; } = string.Empty;

    public DateOnly EnrolledOn { get; set; }
    public DateOnly? WithdrawnOn { get; set; }

    public Guid EnrolmentStatusId { get; set; }
    public string EnrolmentStatusName { get; set; } = string.Empty;

    public string? Notes { get; set; }
}

public class CreateEnrolmentRequest
{
    [Required] public Guid StudentId { get; set; }
    [Required] public Guid SchoolClassId { get; set; }
    [Required] public DateOnly EnrolledOn { get; set; }
```

```
    public Guid? EnrolmentStatusId { get; set; }

    [StringLength(500)]
    public string? Notes { get; set; }
}

public class UpdateEnrolmentRequest
{
    public Guid Id { get; set; }
    [Required] public Guid SchoolClassId { get; set; }
    [Required] public DateOnly EnrolledOn { get; set; }
    public DateOnly? WithdrawnOn { get; set; }
    [Required] public Guid EnrolmentStatusId { get; set; }

    [StringLength(500)]
    public string? Notes { get; set; }
}

public class EnrolmentListFilter
{
    public Guid? StudentId { get; set; }
    public Guid? SessionId { get; set; }
    public Guid? SchoolClassId { get; set; }
    public Guid? EnrolmentStatusId { get; set; }
}
```

Note that CreateEnrolmentRequest has Guid? EnrolmentStatusId — callers who do not specify it pick up the lowest-DisplayOrder status (which is 'Active'), which matches the typical UX where an admin creates a new enrolment and only later realises they wanted a non-default status.

4.6 IStudentService.cs

Listing — src/NaijaPrimeSchool.Application/Family/IStudentService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Family.Dtos;
using NaijaPrimeSchool.Application.Users.Dtos;

namespace NaijaPrimeSchool.Application.Family;

public interface IStudentService
{
    Task<PagedResult<StudentDto>> ListAsync(StudentListFilter filter, CancellationToken ct
= default);
    Task<StudentDto?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult<Guid>> CreateAsync(CreateStudentRequest request, CancellationToken
ct = default);
    Task<OperationResult> UpdateAsync(UpdateStudentRequest request, CancellationToken ct =
default);
    Task<OperationResult> SetActiveAsync(Guid id, bool isActive, CancellationToken ct =
default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);

    Task<IReadOnlyList<StudentParentDto>> GetParentLinksAsync(Guid studentId,
```

```
CancellationToken ct = default);
    Task<OperationResult<Guid>> LinkParentAsync(LinkStudentParentRequest request,
CancellationToken ct = default);
    Task<OperationResult> UpdateParentLinkAsync(UpdateStudentParentLinkRequest request,
CancellationToken ct = default);
    Task<OperationResult> UnlinkParentAsync(Guid linkId, CancellationToken ct = default);
}
```

The student service owns the parent-linkage methods because the linkage is conceptually 'a thing the student has'. There is no IStudentParentService.

4.7 IParentService.cs

Listing — src/NaijaPrimeSchool.Application/Family/IParentService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Family.Dtos;
using NaijaPrimeSchool.Application.Users.Dtos;

namespace NaijaPrimeSchool.Application.Family;

public interface IParentService
{
    Task<PagedResult<ParentDto>> ListAsync(ParentListFilter filter, CancellationToken ct =
default);
    Task<ParentDto?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult<Guid>> CreateAsync(CreateParentRequest request, CancellationToken
ct = default);
    Task<OperationResult> UpdateAsync(UpdateParentRequest request, CancellationToken ct =
default);
    Task<OperationResult> SetActiveAsync(Guid id, bool isActive, CancellationToken ct =
default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);

    Task<IReadOnlyList<StudentParentDto>> GetStudentLinksAsync(Guid parentId,
CancellationToken ct = default);
}
```

4.8 IEnrolmentService.cs

Listing — src/NaijaPrimeSchool.Application/Family/IEnrolmentService.cs

```
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Family.Dtos;

namespace NaijaPrimeSchool.Application.Family;

public interface IEnrolmentService
{
    Task<IReadOnlyList<EnrolmentDto>> ListAsync(EnrolmentListFilter filter,
CancellationToken ct = default);
    Task<EnrolmentDto?> GetByIdAsync(Guid id, CancellationToken ct = default);
    Task<OperationResult<Guid>> CreateAsync(CreateEnrolmentRequest request,
```

```
CancellationToken ct = default);
    Task<OperationResult> UpdateAsync(UpdateEnrolmentRequest request, CancellationToken ct
= default);
    Task<OperationResult> WithdrawAsync(Guid id, DateOnly withdrawnOn, string? notes,
CancellationToken ct = default);
    Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct = default);
}
```

WithdrawAsync is a convenience: it sets WithdrawnOn, flips the status to 'Withdrawn' if that lookup row exists, and appends the supplied notes to the existing notes. The fall-through to UpdateAsync (which can also set WithdrawnOn) gives admins a more flexible path when they need to backdate or correct a record.

4.9 ILookupService extension

ILookupService grew by seven methods. Five new lookups expose the new tables; two are search helpers for the existing Students and Parents tables that the dropdowns on EditStudent lean on:

Listing — src/NaijaPrimeSchool.Application/Users/ILookupService.cs

```
using NaijaPrimeSchool.Application.Users.Dtos;

namespace NaijaPrimeSchool.Application.Users;

public interface ILookupService
{
    Task<IReadOnlyList<LookupDto>> GetGendersAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetTitlesAsync(CancellationToken ct = default);
    Task<IReadOnlyList<RoleDto>> GetRolesAsync(CancellationToken ct = default);

    Task<IReadOnlyList<LookupDto>> GetTermTypesAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetClassLevelsAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetWeekDaysAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetTeachersAsync(CancellationToken ct = default);

    Task<IReadOnlyList<LookupDto>> GetRelationshipsAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetEnrolmentStatusesAsync(CancellationToken ct =
default);
    Task<IReadOnlyList<LookupDto>> GetBloodGroupsAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetMaritalStatusesAsync(CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetClassesForSessionAsync(Guid? sessionId = null,
CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetStudentsAsync(string? searchTerm = null,
CancellationToken ct = default);
    Task<IReadOnlyList<LookupDto>> GetParentsAsync(string? searchTerm = null,
CancellationToken ct = default);
}
```

GetClassesForSessionAsync was strictly speaking a sprint 2-era convenience that never got built — Sprint 3 needs it for the student create form, the enrolment grid, and the family pages, so it lands here. It is technically about Academics but it lives on the same lookup service for UI convenience.

5. Infrastructure — DbContext changes

ApplicationDbContext picks up eight new DbSets and a single new ConfigureFamily method. Everything else in the file is unchanged from sprint 2. The pattern of a private ConfigureXxx method per feature folder keeps OnModelCreating from becoming a thousand-line monster.

5.1 New DbSets

Excerpt — the eight family DbSets

```
    public DbSet<Relationship> Relationships => Set<Relationship>();
    public DbSet<EnrolmentStatus> EnrolmentStatuses => Set<EnrolmentStatus>();
    public DbSet<BloodGroup> BloodGroups => Set<BloodGroup>();
    public DbSet<MaritalStatus> MaritalStatuses => Set<MaritalStatus>();
    public DbSet<Student> Students => Set<Student>();
    public DbSet<Parent> Parents => Set<Parent>();
    public DbSet<StudentParent> StudentParents => Set<StudentParent>();
    public DbSet<Enrolment> Enrolments => Set<Enrolment>();
```

5.2 ConfigureFamily

ConfigureFamily is invoked from OnModelCreating after ConfigureAcademics. It uses the same ConfigureLookup helper from sprint 1 for the four lookup tables, then configures Student, Parent, StudentParent, and Enrolment one by one.

Excerpt — ConfigureFamily

```
    private static void ConfigureFamily(ModelBuilder builder)
    {
        ConfigureLookup<Relationship>(builder, "Relationships", extra: b =>
        {
            b.Property(r => r.Name).HasMaxLength(40).IsRequired();
            b.HasIndex(r => r.Name).IsUnique();
        });

        ConfigureLookup<EnrolmentStatus>(builder, "EnrolmentStatuses", extra: b =>
        {
            b.Property(s => s.Name).HasMaxLength(40).IsRequired();
            b.HasIndex(s => s.Name).IsUnique();
        });

        ConfigureLookup<BloodGroup>(builder, "BloodGroups", extra: b =>
        {
            b.Property(g => g.Name).HasMaxLength(10).IsRequired();
            b.HasIndex(g => g.Name).IsUnique();
        });

        ConfigureLookup<MaritalStatus>(builder, "MaritalStatuses", extra: b =>
        {
            b.Property(m => m.Name).HasMaxLength(40).IsRequired();
            b.HasIndex(m => m.Name).IsUnique();
        });

        builder.Entity<Student>(b =>
```

```
        {
            b.ToTable("Students");
            b.HasKey(s => s.Id);
            b.Property(s => s.AdmissionNumber).HasMaxLength(30).IsRequired();
            b.Property(s => s.FirstName).HasMaxLength(80).IsRequired();
            b.Property(s => s.LastName).HasMaxLength(80).IsRequired();
            b.Property(s => s.MiddleName).HasMaxLength(80);
            b.Property(s => s.StateOfOrigin).HasMaxLength(80);
            b.Property(s => s.ResidentialAddress).HasMaxLength(300);
            b.Property(s => s.PhotoUrl).HasMaxLength(500);
            b.Property(s => s.Allergies).HasMaxLength(500);
            b.Property(s => s.MedicalNotes).HasMaxLength(1000);
            b.Property(s => s.CreatedBy).HasMaxLength(100);
            b.Property(s => s.ModifiedBy).HasMaxLength(100);
            b.Property(s => s.DeletedBy).HasMaxLength(100);
            b.Ignore(s => s.FullName);

            b.HasOne(s => s.Gender).WithMany()
                .HasForeignKey(s => s.GenderId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(s => s.BloodGroup).WithMany(g => g.Students)
                .HasForeignKey(s => s.BloodGroupId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(s => s.User).WithMany()
                .HasForeignKey(s => s.UserId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasIndex(s => s.AdmissionNumber).IsUnique();
            b.HasIndex(s => s.UserId).IsUnique()
                .HasFilter("[UserId] IS NOT NULL");
            b.HasIndex(s => s.IsDeleted);
            b.HasQueryFilter(s => !s.IsDeleted);
        });

        builder.Entity<Parent>(b =>
        {
            b.ToTable("Parents");
            b.HasKey(p => p.Id);
            b.Property(p => p.FirstName).HasMaxLength(80).IsRequired();
            b.Property(p => p.LastName).HasMaxLength(80).IsRequired();
            b.Property(p => p.MiddleName).HasMaxLength(80);
            b.Property(p => p.PrimaryPhone).HasMaxLength(30);
            b.Property(p => p.AlternatePhone).HasMaxLength(30);
            b.Property(p => p.Email).HasMaxLength(256);
            b.Property(p => p.ResidentialAddress).HasMaxLength(300);
            b.Property(p => p.Occupation).HasMaxLength(120);
            b.Property(p => p.Employer).HasMaxLength(120);
            b.Property(p => p.CreatedBy).HasMaxLength(100);
            b.Property(p => p.ModifiedBy).HasMaxLength(100);
            b.Property(p => p.DeletedBy).HasMaxLength(100);
            b.Ignore(p => p.FullName);
```

```
            b.HasOne(p => p.Title).WithMany()
                .HasForeignKey(p => p.TitleId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(p => p.Gender).WithMany()
                .HasForeignKey(p => p.GenderId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(p => p.MaritalStatus).WithMany(m => m.Parents)
                .HasForeignKey(p => p.MaritalStatusId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasOne(p => p.User).WithMany()
                .HasForeignKey(p => p.UserId)
                .OnDelete(DeleteBehavior.SetNull);

            b.HasIndex(p => p.UserId).IsUnique()
                .HasFilter("[UserId] IS NOT NULL");
            b.HasIndex(p => p.IsDeleted);
            b.HasQueryFilter(p => !p.IsDeleted);
        });

        builder.Entity<StudentParent>(b =>
        {
            b.ToTable("StudentParents");
            b.HasKey(sp => sp.Id);
            b.Property(sp => sp.Notes).HasMaxLength(300);
            b.Property(sp => sp.CreatedBy).HasMaxLength(100);
            b.Property(sp => sp.ModifiedBy).HasMaxLength(100);
            b.Property(sp => sp.DeletedBy).HasMaxLength(100);

            b.HasOne(sp => sp.Student).WithMany(s => s.ParentLinks)
                .HasForeignKey(sp => sp.StudentId)
                .OnDelete(DeleteBehavior.Cascade);

            b.HasOne(sp => sp.Parent).WithMany(p => p.StudentLinks)
                .HasForeignKey(sp => sp.ParentId)
                .OnDelete(DeleteBehavior.Cascade);

            b.HasOne(sp => sp.Relationship).WithMany(r => r.StudentLinks)
                .HasForeignKey(sp => sp.RelationshipId)
                .OnDelete(DeleteBehavior.Restrict);

            // A given parent should appear at most once per student.
            b.HasIndex(sp => new { sp.StudentId, sp.ParentId }).IsUnique();
            b.HasIndex(sp => sp.IsDeleted);
            b.HasQueryFilter(sp => !sp.IsDeleted);
        });

        builder.Entity<Enrolment>(b =>
        {
            b.ToTable("Enrolments");
            b.HasKey(e => e.Id);
            b.Property(e => e.Notes).HasMaxLength(500);
```

```
            b.Property(e => e.CreatedBy).HasMaxLength(100);
            b.Property(e => e.ModifiedBy).HasMaxLength(100);
            b.Property(e => e.DeletedBy).HasMaxLength(100);

            b.HasOne(e => e.Student).WithMany(s => s.Enrolments)
                .HasForeignKey(e => e.StudentId)
                .OnDelete(DeleteBehavior.Cascade);

            b.HasOne(e => e.SchoolClass).WithMany(c => c.Enrolments)
                .HasForeignKey(e => e.SchoolClassId)
                .OnDelete(DeleteBehavior.Restrict);

            b.HasOne(e => e.EnrolmentStatus).WithMany(s => s.Enrolments)
                .HasForeignKey(e => e.EnrolmentStatusId)
                .OnDelete(DeleteBehavior.Restrict);

            // A student appears at most once in a given class.
            b.HasIndex(e => new { e.StudentId, e.SchoolClassId }).IsUnique();
            b.HasIndex(e => e.IsDeleted);
            b.HasQueryFilter(e => !e.IsDeleted);
        });
    }

    private static void ConfigureLookup<TEntity>(
```

Three things are worth dwelling on here:

- b.Ignore(s => s.FullName) on Student and Parent — the computed property is a string, so EF Core would otherwise try to map it as a column.
- b.HasIndex(s => s.UserId).IsUnique().HasFilter("[UserId] IS NOT NULL") — a filtered unique index that allows many rows to have a null UserId today, but at most one row to point at a given ApplicationUser when the portal sprint hooks them up.
- DeleteBehavior.Cascade on the StudentParent and Enrolment foreign keys back to Student. Re-read the design notes in 2.7 for why; the short version is that hard-delete via a script should be self-cleaning.

5.3 The lookup helper, reused

The ConfigureLookup<T> generic helper from sprint 1 is reused unchanged. Each new lookup picks up table name, primary key, audit columns, and the global query filter on IsDeleted — the only customisation per lookup is the Name column constraint and a unique index.

Excerpt — ConfigureLookup<T> (unchanged from sprint 1)

```
    private static void ConfigureLookup<TEntity>(
        ModelBuilder builder,
        string tableName,
        Action<EntityTypeBuilder<TEntity>>? extra = null) where TEntity : BaseEntity
    {
        builder.Entity<TEntity>(b =>
        {
            b.ToTable(tableName);
```

```
            b.HasKey(e => e.Id);
            b.Property(e => e.CreatedBy).HasMaxLength(100);
            b.Property(e => e.ModifiedBy).HasMaxLength(100);
            b.Property(e => e.DeletedBy).HasMaxLength(100);
            b.HasQueryFilter(BuildIsNotDeletedExpression<TEntity>());
            extra?.Invoke(b);
        });
    }

    private static Expression<Func<TEntity, bool>> BuildIsNotDeletedExpression<TEntity>()
        where TEntity : ISoftDelete
    {
        return e => !e.IsDeleted;
    }

    public override int SaveChanges()
```

5.4 SaveChanges still does all the work

The SaveChanges override from sprint 1 needs no changes for sprint 3. Every new entity inherits BaseEntity, so the audit stamping and soft-delete rewriting apply automatically. This is the single biggest payoff of the architectural decisions in sprint 1: every sprint after it gets auditing and soft delete for free.

6. Infrastructure — service implementations

Three new services land in src/NaijaPrimeSchool.Infrastructure/Services/, each one a straightforward implementation of the matching Application interface. Plus the existing LookupService grows seven methods to surface the new lookup tables.

6.1 StudentService.cs

StudentService is the heaviest of the three. It owns the pupil CRUD, the parent-linkage CRUD, and the student-level primitives (search, list, set-active, soft-delete).

Listing — src/NaijaPrimeSchool.Infrastructure/Services/StudentService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Family;
using NaijaPrimeSchool.Application.Family.Dtos;
using NaijaPrimeSchool.Application.Users.Dtos;
using NaijaPrimeSchool.Domain.Family;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class StudentService(ApplicationDbContext db) : IStudentService
{
    public async Task<PagedResult<StudentDto>> ListAsync(StudentListFilter filter,
CancellationToken ct = default)
    {
        var query = db.Students.AsQueryable();

        if (!string.IsNullOrWhiteSpace(filter.SearchTerm))
        {
            var term = filter.SearchTerm.Trim().ToLower();
            query = query.Where(s =>
                s.FirstName.ToLower().Contains(term) ||
                s.LastName.ToLower().Contains(term) ||
                (s.MiddleName != null && s.MiddleName.ToLower().Contains(term)) ||
                s.AdmissionNumber.ToLower().Contains(term));
        }

        if (filter.IsActive.HasValue)
        {
            query = query.Where(s => s.IsActive == filter.IsActive.Value);
        }

        if (filter.ClassId.HasValue)
        {
            var classId = filter.ClassId.Value;
            query = query.Where(s =>
                s.Enrolments.Any(e => e.SchoolClassId == classId && e.WithdrawnOn ==
null));
        }

        if (filter.SessionId.HasValue)
        {
```

```
            var sessionId = filter.SessionId.Value;
            query = query.Where(s =>
                s.Enrolments.Any(e => e.SchoolClass!.SessionId == sessionId));
        }

        var total = await query.CountAsync(ct);

        query = filter.OrderBy?.ToLower() switch
        {
            "name desc" => query.OrderByDescending(s => s.FirstName).ThenByDescending(s =>
s.LastName),
            "admission asc" => query.OrderBy(s => s.AdmissionNumber),
            "admission desc" => query.OrderByDescending(s => s.AdmissionNumber),
            "createdon desc" => query.OrderByDescending(s => s.CreatedOn),
            _ => query.OrderBy(s => s.FirstName).ThenBy(s => s.LastName),
        };

        var items = await query
            .Skip(filter.Skip)
            .Take(filter.Take)
            .Select(s => new StudentDto
            {
                Id = s.Id,
                AdmissionNumber = s.AdmissionNumber,
                AdmissionDate = s.AdmissionDate,
                FirstName = s.FirstName,
                LastName = s.LastName,
                MiddleName = s.MiddleName,
                FullName = s.FirstName + (s.MiddleName == null ? "" : " " + s.MiddleName) +
" " + s.LastName,
                DateOfBirth = s.DateOfBirth,
                GenderId = s.GenderId,
                GenderName = s.Gender == null ? null : s.Gender.Name,
                BloodGroupId = s.BloodGroupId,
                BloodGroupName = s.BloodGroup == null ? null : s.BloodGroup.Name,
                StateOfOrigin = s.StateOfOrigin,
                ResidentialAddress = s.ResidentialAddress,
                PhotoUrl = s.PhotoUrl,
                Allergies = s.Allergies,
                MedicalNotes = s.MedicalNotes,
                IsActive = s.IsActive,
                CurrentClassId = s.Enrolments
                    .Where(e => e.WithdrawnOn == null)
                    .OrderByDescending(e => e.EnrolledOn)
                    .Select(e => (Guid?)e.SchoolClassId)
                    .FirstOrDefault(),
                CurrentClassName = s.Enrolments
                    .Where(e => e.WithdrawnOn == null)
                    .OrderByDescending(e => e.EnrolledOn)
                    .Select(e => e.SchoolClass!.Name)
                    .FirstOrDefault(),
                CurrentSessionId = s.Enrolments
                    .Where(e => e.WithdrawnOn == null)
                    .OrderByDescending(e => e.EnrolledOn)
                    .Select(e => (Guid?)e.SchoolClass!.SessionId)
```

```
                    .FirstOrDefault(),
                CurrentSessionName = s.Enrolments
                    .Where(e => e.WithdrawnOn == null)
                    .OrderByDescending(e => e.EnrolledOn)
                    .Select(e => e.SchoolClass!.Session!.Name)
                    .FirstOrDefault(),
                ParentCount = s.ParentLinks.Count,
                PrimaryContactName = s.ParentLinks
                    .Where(l => l.IsPrimaryContact)
                    .Select(l => l.Parent!.FirstName + " " + l.Parent!.LastName)
                    .FirstOrDefault(),
                PrimaryContactPhone = s.ParentLinks
                    .Where(l => l.IsPrimaryContact)
                    .Select(l => l.Parent!.PrimaryPhone)
                    .FirstOrDefault(),
            })
            .ToListAsync(ct);

        return new PagedResult<StudentDto> { Items = items, TotalCount = total };
    }

    public Task<StudentDto?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        db.Students.Where(s => s.Id == id)
            .Select(s => new StudentDto
            {
                Id = s.Id,
                AdmissionNumber = s.AdmissionNumber,
                AdmissionDate = s.AdmissionDate,
                FirstName = s.FirstName,
                LastName = s.LastName,
                MiddleName = s.MiddleName,
                FullName = s.FirstName + (s.MiddleName == null ? "" : " " + s.MiddleName) +
" " + s.LastName,
                DateOfBirth = s.DateOfBirth,
                GenderId = s.GenderId,
                GenderName = s.Gender == null ? null : s.Gender.Name,
                BloodGroupId = s.BloodGroupId,
                BloodGroupName = s.BloodGroup == null ? null : s.BloodGroup.Name,
                StateOfOrigin = s.StateOfOrigin,
                ResidentialAddress = s.ResidentialAddress,
                PhotoUrl = s.PhotoUrl,
                Allergies = s.Allergies,
                MedicalNotes = s.MedicalNotes,
                IsActive = s.IsActive,
                CurrentClassId = s.Enrolments
                    .Where(e => e.WithdrawnOn == null)
                    .OrderByDescending(e => e.EnrolledOn)
                    .Select(e => (Guid?)e.SchoolClassId)
                    .FirstOrDefault(),
                CurrentClassName = s.Enrolments
                    .Where(e => e.WithdrawnOn == null)
                    .OrderByDescending(e => e.EnrolledOn)
                    .Select(e => e.SchoolClass!.Name)
                    .FirstOrDefault(),
                CurrentSessionId = s.Enrolments
```

```
                    .Where(e => e.WithdrawnOn == null)
                    .OrderByDescending(e => e.EnrolledOn)
                    .Select(e => (Guid?)e.SchoolClass!.SessionId)
                    .FirstOrDefault(),
                CurrentSessionName = s.Enrolments
                    .Where(e => e.WithdrawnOn == null)
                    .OrderByDescending(e => e.EnrolledOn)
                    .Select(e => e.SchoolClass!.Session!.Name)
                    .FirstOrDefault(),
                ParentCount = s.ParentLinks.Count,
            })
            .FirstOrDefaultAsync(ct);

    public async Task<OperationResult<Guid>> CreateAsync(CreateStudentRequest request,
CancellationToken ct = default)
    {
        if (request.DateOfBirth >= request.AdmissionDate)
            return OperationResult<Guid>.Failure("Date of birth must be earlier than the
admission date.");

        if (await db.Students.AnyAsync(s => s.AdmissionNumber == request.AdmissionNumber,
ct))
            return OperationResult<Guid>.Failure(
                $"A student with admission number '{request.AdmissionNumber}' already
exists.");

        if (request.InitialClassId.HasValue
            && !await db.SchoolClasses.AnyAsync(c => c.Id == request.InitialClassId.Value,
ct))
            return OperationResult<Guid>.Failure("Selected class does not exist.");

        var student = new Student
        {
            AdmissionNumber = request.AdmissionNumber.Trim(),
            AdmissionDate = request.AdmissionDate,
            FirstName = request.FirstName.Trim(),
            LastName = request.LastName.Trim(),
            MiddleName = string.IsNullOrWhiteSpace(request.MiddleName) ? null :
request.MiddleName.Trim(),
            DateOfBirth = request.DateOfBirth,
            GenderId = request.GenderId,
            BloodGroupId = request.BloodGroupId,
            StateOfOrigin = request.StateOfOrigin,
            ResidentialAddress = request.ResidentialAddress,
            PhotoUrl = request.PhotoUrl,
            Allergies = request.Allergies,
            MedicalNotes = request.MedicalNotes,
            IsActive = request.IsActive,
        };

        db.Students.Add(student);

        if (request.InitialClassId.HasValue)
        {
```

```
            var defaultStatus = await db.EnrolmentStatuses
                .OrderBy(s => s.DisplayOrder)
                .FirstOrDefaultAsync(ct);

            if (defaultStatus is null)
                return OperationResult<Guid>.Failure("Enrolment statuses are not seeded.");

            db.Enrolments.Add(new Enrolment
            {
                Student = student,
                SchoolClassId = request.InitialClassId.Value,
                EnrolledOn = request.AdmissionDate,
                EnrolmentStatusId = defaultStatus.Id,
            });
        }

        await db.SaveChangesAsync(ct);
        return OperationResult<Guid>.Success(student.Id);
    }

    public async Task<OperationResult> UpdateAsync(UpdateStudentRequest request,
CancellationToken ct = default)
    {
        var student = await db.Students.FirstOrDefaultAsync(s => s.Id == request.Id, ct);
        if (student is null) return OperationResult.Failure("Student not found.");

        if (request.DateOfBirth >= request.AdmissionDate)
            return OperationResult.Failure("Date of birth must be earlier than the
admission date.");

        if (await db.Students.AnyAsync(s => s.AdmissionNumber == request.AdmissionNumber &&
s.Id != request.Id, ct))
            return OperationResult.Failure(
                $"A student with admission number '{request.AdmissionNumber}' already
exists.");

        student.AdmissionNumber = request.AdmissionNumber.Trim();
        student.AdmissionDate = request.AdmissionDate;
        student.FirstName = request.FirstName.Trim();
        student.LastName = request.LastName.Trim();
        student.MiddleName = string.IsNullOrWhiteSpace(request.MiddleName) ? null :
request.MiddleName.Trim();
        student.DateOfBirth = request.DateOfBirth;
        student.GenderId = request.GenderId;
        student.BloodGroupId = request.BloodGroupId;
        student.StateOfOrigin = request.StateOfOrigin;
        student.ResidentialAddress = request.ResidentialAddress;
        student.PhotoUrl = request.PhotoUrl;
        student.Allergies = request.Allergies;
        student.MedicalNotes = request.MedicalNotes;

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }
```

```
    public async Task<OperationResult> SetActiveAsync(Guid id, bool isActive,
CancellationToken ct = default)
    {
        var student = await db.Students.FirstOrDefaultAsync(s => s.Id == id, ct);
        if (student is null) return OperationResult.Failure("Student not found.");

        student.IsActive = isActive;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
        var student = await db.Students
            .Include(s => s.Enrolments)
            .Include(s => s.ParentLinks)
            .FirstOrDefaultAsync(s => s.Id == id, ct);

        if (student is null) return OperationResult.Failure("Student not found.");

        if (student.Enrolments.Any(e => e.WithdrawnOn == null))
            return OperationResult.Failure(
                "Cannot delete a student with an active enrolment. Withdraw the enrolment
first.");

        db.Students.Remove(student);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public Task<IReadOnlyList<StudentParentDto>> GetParentLinksAsync(Guid studentId,
CancellationToken ct = default) =>
        ProjectLinks(db.StudentParents.Where(l => l.StudentId == studentId), ct);

    public async Task<OperationResult<Guid>> LinkParentAsync(LinkStudentParentRequest
request, CancellationToken ct = default)
    {
        if (!await db.Students.AnyAsync(s => s.Id == request.StudentId, ct))
            return OperationResult<Guid>.Failure("Student not found.");

        if (!await db.Parents.AnyAsync(p => p.Id == request.ParentId, ct))
            return OperationResult<Guid>.Failure("Parent not found.");

        if (!await db.Relationships.AnyAsync(r => r.Id == request.RelationshipId, ct))
            return OperationResult<Guid>.Failure("Relationship is not valid.");

        if (await db.StudentParents.AnyAsync(l =>
                l.StudentId == request.StudentId && l.ParentId == request.ParentId, ct))
            return OperationResult<Guid>.Failure("This parent is already linked to the
student.");

        if (request.IsPrimaryContact)
```

```
        {
            await db.StudentParents
                .Where(l => l.StudentId == request.StudentId && l.IsPrimaryContact)
                .ExecuteUpdateAsync(s => s.SetProperty(l => l.IsPrimaryContact, false),
ct);
        }

        var link = new StudentParent
        {
            StudentId = request.StudentId,
            ParentId = request.ParentId,
            RelationshipId = request.RelationshipId,
            IsPrimaryContact = request.IsPrimaryContact,
            CanPickUp = request.CanPickUp,
            Notes = request.Notes,
        };
        db.StudentParents.Add(link);
        await db.SaveChangesAsync(ct);
        return OperationResult<Guid>.Success(link.Id);
    }

    public async Task<OperationResult> UpdateParentLinkAsync(UpdateStudentParentLinkRequest
request, CancellationToken ct = default)
    {
        var link = await db.StudentParents.FirstOrDefaultAsync(l => l.Id == request.Id,
ct);
        if (link is null) return OperationResult.Failure("Link not found.");

        if (!await db.Relationships.AnyAsync(r => r.Id == request.RelationshipId, ct))
            return OperationResult.Failure("Relationship is not valid.");

        if (request.IsPrimaryContact && !link.IsPrimaryContact)
        {
            await db.StudentParents
                .Where(l => l.StudentId == link.StudentId && l.IsPrimaryContact && l.Id !=
link.Id)
                .ExecuteUpdateAsync(s => s.SetProperty(l => l.IsPrimaryContact, false),
ct);
        }

        link.RelationshipId = request.RelationshipId;
        link.IsPrimaryContact = request.IsPrimaryContact;
        link.CanPickUp = request.CanPickUp;
        link.Notes = request.Notes;

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> UnlinkParentAsync(Guid linkId, CancellationToken ct
= default)
    {
        var link = await db.StudentParents.FirstOrDefaultAsync(l => l.Id == linkId, ct);
        if (link is null) return OperationResult.Failure("Link not found.");
```

```
        db.StudentParents.Remove(link);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    private static async Task<IReadOnlyList<StudentParentDto>> ProjectLinks(
        IQueryable<StudentParent> q, CancellationToken ct) =>
        await q.OrderByDescending(l => l.IsPrimaryContact)
            .ThenBy(l => l.Parent!.FirstName)
            .Select(l => new StudentParentDto
            {
                Id = l.Id,
                StudentId = l.StudentId,
                StudentName = (l.Student!.FirstName + " " + l.Student!.LastName).Trim(),
                StudentAdmissionNumber = l.Student!.AdmissionNumber,
                ParentId = l.ParentId,
                ParentName = (l.Parent!.FirstName + " " + l.Parent!.LastName).Trim(),
                ParentPhone = l.Parent!.PrimaryPhone,
                ParentEmail = l.Parent!.Email,
                RelationshipId = l.RelationshipId,
                RelationshipName = l.Relationship!.Name,
                IsPrimaryContact = l.IsPrimaryContact,
                CanPickUp = l.CanPickUp,
                Notes = l.Notes,
            })
            .ToListAsync(ct);
}
```

Three patterns are worth calling out from this file:

- ListAsync uses a single big projection that pulls CurrentClass, CurrentSession, ParentCount, and PrimaryContactName from subqueries. The whole list page is one round-trip to the database, with no N+1 queries even when the filters narrow it to a thousand pupils.
- CreateAsync optionally creates an enrolment row in the same SaveChanges. The enrolment uses the student's AdmissionDate as the EnrolledOn — admins almost always enrol on the day of admission, and the form gives them the AdmissionDate field anyway.
- LinkParentAsync flips IsPrimaryContact = false on every other link for the same student before saving — there can be at most one primary contact per pupil, and the service enforces that with a single ExecuteUpdateAsync rather than reading and writing in a loop.

6.2 ParentService.cs

ParentService is parallel to StudentService but simpler — there is no enrolment side. The grouping pattern (search, list, get-by-id, create, update, set-active, soft-delete) is the same.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/ParentService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Family;
using NaijaPrimeSchool.Application.Family.Dtos;
```

```
using NaijaPrimeSchool.Application.Users.Dtos;
using NaijaPrimeSchool.Domain.Family;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class ParentService(ApplicationDbContext db) : IParentService
{
    public async Task<PagedResult<ParentDto>> ListAsync(ParentListFilter filter,
CancellationToken ct = default)
    {
        var query = db.Parents.AsQueryable();

        if (!string.IsNullOrWhiteSpace(filter.SearchTerm))
        {
            var term = filter.SearchTerm.Trim().ToLower();
            query = query.Where(p =>
                p.FirstName.ToLower().Contains(term) ||
                p.LastName.ToLower().Contains(term) ||
                (p.MiddleName != null && p.MiddleName.ToLower().Contains(term)) ||
                (p.PrimaryPhone != null && p.PrimaryPhone.Contains(term)) ||
                (p.AlternatePhone != null && p.AlternatePhone.Contains(term)) ||
                (p.Email != null && p.Email.ToLower().Contains(term)));
        }

        if (filter.IsActive.HasValue)
        {
            query = query.Where(p => p.IsActive == filter.IsActive.Value);
        }

        var total = await query.CountAsync(ct);

        query = filter.OrderBy?.ToLower() switch
        {
            "name desc" => query.OrderByDescending(p => p.FirstName).ThenByDescending(p =>
p.LastName),
            "createdon desc" => query.OrderByDescending(p => p.CreatedOn),
            _ => query.OrderBy(p => p.FirstName).ThenBy(p => p.LastName),
        };

        var items = await query
            .Skip(filter.Skip)
            .Take(filter.Take)
            .Select(p => new ParentDto
            {
                Id = p.Id,
                FirstName = p.FirstName,
                LastName = p.LastName,
                MiddleName = p.MiddleName,
                FullName = p.FirstName + (p.MiddleName == null ? "" : " " + p.MiddleName) +
" " + p.LastName,
                TitleId = p.TitleId,
                TitleName = p.Title == null ? null : p.Title.Name,
                GenderId = p.GenderId,
```

```
                GenderName = p.Gender == null ? null : p.Gender.Name,
                MaritalStatusId = p.MaritalStatusId,
                MaritalStatusName = p.MaritalStatus == null ? null : p.MaritalStatus.Name,
                PrimaryPhone = p.PrimaryPhone,
                AlternatePhone = p.AlternatePhone,
                Email = p.Email,
                ResidentialAddress = p.ResidentialAddress,
                Occupation = p.Occupation,
                Employer = p.Employer,
                IsActive = p.IsActive,
                StudentCount = p.StudentLinks.Count,
            })
            .ToListAsync(ct);

        return new PagedResult<ParentDto> { Items = items, TotalCount = total };
    }

    public Task<ParentDto?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        db.Parents.Where(p => p.Id == id)
            .Select(p => new ParentDto
            {
                Id = p.Id,
                FirstName = p.FirstName,
                LastName = p.LastName,
                MiddleName = p.MiddleName,
                FullName = p.FirstName + (p.MiddleName == null ? "" : " " + p.MiddleName) +
" " + p.LastName,
                TitleId = p.TitleId,
                TitleName = p.Title == null ? null : p.Title.Name,
                GenderId = p.GenderId,
                GenderName = p.Gender == null ? null : p.Gender.Name,
                MaritalStatusId = p.MaritalStatusId,
                MaritalStatusName = p.MaritalStatus == null ? null : p.MaritalStatus.Name,
                PrimaryPhone = p.PrimaryPhone,
                AlternatePhone = p.AlternatePhone,
                Email = p.Email,
                ResidentialAddress = p.ResidentialAddress,
                Occupation = p.Occupation,
                Employer = p.Employer,
                IsActive = p.IsActive,
                StudentCount = p.StudentLinks.Count,
            })
            .FirstOrDefaultAsync(ct);

    public async Task<OperationResult<Guid>> CreateAsync(CreateParentRequest request,
CancellationToken ct = default)
    {
        var parent = new Parent
        {
            FirstName = request.FirstName.Trim(),
            LastName = request.LastName.Trim(),
            MiddleName = string.IsNullOrWhiteSpace(request.MiddleName) ? null :
request.MiddleName.Trim(),
            TitleId = request.TitleId,
            GenderId = request.GenderId,
```

```
            MaritalStatusId = request.MaritalStatusId,
            PrimaryPhone = request.PrimaryPhone,
            AlternatePhone = request.AlternatePhone,
            Email = request.Email,
            ResidentialAddress = request.ResidentialAddress,
            Occupation = request.Occupation,
            Employer = request.Employer,
            IsActive = request.IsActive,
        };
        db.Parents.Add(parent);
        await db.SaveChangesAsync(ct);
        return OperationResult<Guid>.Success(parent.Id);
    }

    public async Task<OperationResult> UpdateAsync(UpdateParentRequest request,
CancellationToken ct = default)
    {
        var parent = await db.Parents.FirstOrDefaultAsync(p => p.Id == request.Id, ct);
        if (parent is null) return OperationResult.Failure("Parent not found.");

        parent.FirstName = request.FirstName.Trim();
        parent.LastName = request.LastName.Trim();
        parent.MiddleName = string.IsNullOrWhiteSpace(request.MiddleName) ? null :
request.MiddleName.Trim();
        parent.TitleId = request.TitleId;
        parent.GenderId = request.GenderId;
        parent.MaritalStatusId = request.MaritalStatusId;
        parent.PrimaryPhone = request.PrimaryPhone;
        parent.AlternatePhone = request.AlternatePhone;
        parent.Email = request.Email;
        parent.ResidentialAddress = request.ResidentialAddress;
        parent.Occupation = request.Occupation;
        parent.Employer = request.Employer;

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SetActiveAsync(Guid id, bool isActive,
CancellationToken ct = default)
    {
        var parent = await db.Parents.FirstOrDefaultAsync(p => p.Id == id, ct);
        if (parent is null) return OperationResult.Failure("Parent not found.");

        parent.IsActive = isActive;
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
        var parent = await db.Parents
            .Include(p => p.StudentLinks)
```

```
            .FirstOrDefaultAsync(p => p.Id == id, ct);

        if (parent is null) return OperationResult.Failure("Parent not found.");

        if (parent.StudentLinks.Count > 0)
            return OperationResult.Failure(
                "Cannot delete a parent who is still linked to a student. Unlink them
first.");

        db.Parents.Remove(parent);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public Task<IReadOnlyList<StudentParentDto>> GetStudentLinksAsync(Guid parentId,
CancellationToken ct = default) =>
        ProjectLinks(db.StudentParents.Where(l => l.ParentId == parentId), ct);

    private static async Task<IReadOnlyList<StudentParentDto>> ProjectLinks(
        IQueryable<StudentParent> q, CancellationToken ct) =>
        await q.OrderBy(l => l.Student!.FirstName)
            .Select(l => new StudentParentDto
            {
                Id = l.Id,
                StudentId = l.StudentId,
                StudentName = (l.Student!.FirstName + " " + l.Student!.LastName).Trim(),
                StudentAdmissionNumber = l.Student!.AdmissionNumber,
                ParentId = l.ParentId,
                ParentName = (l.Parent!.FirstName + " " + l.Parent!.LastName).Trim(),
                ParentPhone = l.Parent!.PrimaryPhone,
                ParentEmail = l.Parent!.Email,
                RelationshipId = l.RelationshipId,
                RelationshipName = l.Relationship!.Name,
                IsPrimaryContact = l.IsPrimaryContact,
                CanPickUp = l.CanPickUp,
                Notes = l.Notes,
            })
            .ToListAsync(ct);
}
```

GetStudentLinksAsync mirrors GetParentLinksAsync on StudentService — both project from the StudentParent table to StudentParentDto, just with a different filter. The two pieces of code are similar enough that they could share a helper, but inlining each one keeps the read-flow explicit and the LINQ short.

6.3 EnrolmentService.cs

EnrolmentService is the smallest, and it deliberately stays narrow. There is no ListAsync paging — enrolments are usually filtered down to a single class or a single pupil before rendering, so 25-row paging on the grid is enough.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/EnrolmentService.cs

```
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Common;
using NaijaPrimeSchool.Application.Family;
using NaijaPrimeSchool.Application.Family.Dtos;
using NaijaPrimeSchool.Domain.Family;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class EnrolmentService(ApplicationDbContext db) : IEnrolmentService
{
    private static IQueryable<EnrolmentDto> Project(IQueryable<Enrolment> q) =>
        q.Select(e => new EnrolmentDto
        {
            Id = e.Id,
            StudentId = e.StudentId,
            StudentName = (e.Student!.FirstName + " " + e.Student!.LastName).Trim(),
            StudentAdmissionNumber = e.Student!.AdmissionNumber,
            SchoolClassId = e.SchoolClassId,
            SchoolClassName = e.SchoolClass!.Name,
            SessionId = e.SchoolClass!.SessionId,
            SessionName = e.SchoolClass!.Session!.Name,
            EnrolledOn = e.EnrolledOn,
            WithdrawnOn = e.WithdrawnOn,
            EnrolmentStatusId = e.EnrolmentStatusId,
            EnrolmentStatusName = e.EnrolmentStatus!.Name,
            Notes = e.Notes,
        });

    public async Task<IReadOnlyList<EnrolmentDto>> ListAsync(EnrolmentListFilter filter,
CancellationToken ct = default)
    {
        var q = db.Enrolments.AsQueryable();

        if (filter.StudentId.HasValue) q = q.Where(e => e.StudentId ==
filter.StudentId.Value);
        if (filter.SchoolClassId.HasValue) q = q.Where(e => e.SchoolClassId ==
filter.SchoolClassId.Value);
        if (filter.SessionId.HasValue) q = q.Where(e => e.SchoolClass!.SessionId ==
filter.SessionId.Value);
        if (filter.EnrolmentStatusId.HasValue) q = q.Where(e => e.EnrolmentStatusId ==
filter.EnrolmentStatusId.Value);

        return await Project(q.OrderByDescending(e => e.EnrolledOn))
            .ToListAsync(ct);
    }

    public Task<EnrolmentDto?> GetByIdAsync(Guid id, CancellationToken ct = default) =>
        Project(db.Enrolments.Where(e => e.Id == id)).FirstOrDefaultAsync(ct);

    public async Task<OperationResult<Guid>> CreateAsync(CreateEnrolmentRequest request,
CancellationToken ct = default)
    {
        if (!await db.Students.AnyAsync(s => s.Id == request.StudentId, ct))
```

```
            return OperationResult<Guid>.Failure("Student not found.");

        var schoolClass = await db.SchoolClasses
            .FirstOrDefaultAsync(c => c.Id == request.SchoolClassId, ct);
        if (schoolClass is null)
            return OperationResult<Guid>.Failure("Class not found.");

        if (await db.Enrolments.AnyAsync(e =>
                e.StudentId == request.StudentId
                && e.SchoolClassId == request.SchoolClassId, ct))
            return OperationResult<Guid>.Failure("Student is already enrolled in this
class.");

        // Within a session a student can be in at most one class at a time.
        var sessionId = schoolClass.SessionId;
        var hasOpenInSession = await db.Enrolments.AnyAsync(e =>
            e.StudentId == request.StudentId
            && e.SchoolClass!.SessionId == sessionId
            && e.WithdrawnOn == null, ct);
        if (hasOpenInSession)
            return OperationResult<Guid>.Failure(
                "Student already has an active enrolment in this session. Withdraw it
first.");

        Guid statusId;
        if (request.EnrolmentStatusId.HasValue)
        {
            if (!await db.EnrolmentStatuses.AnyAsync(s => s.Id ==
request.EnrolmentStatusId.Value, ct))
                return OperationResult<Guid>.Failure("Enrolment status is not valid.");
            statusId = request.EnrolmentStatusId.Value;
        }
        else
        {
            var defaultStatus = await db.EnrolmentStatuses
                .OrderBy(s => s.DisplayOrder)
                .FirstOrDefaultAsync(ct);
            if (defaultStatus is null)
                return OperationResult<Guid>.Failure("Enrolment statuses are not seeded.");
            statusId = defaultStatus.Id;
        }

        var enrolment = new Enrolment
        {
            StudentId = request.StudentId,
            SchoolClassId = request.SchoolClassId,
            EnrolledOn = request.EnrolledOn,
            EnrolmentStatusId = statusId,
            Notes = request.Notes,
        };
        db.Enrolments.Add(enrolment);
        await db.SaveChangesAsync(ct);
        return OperationResult<Guid>.Success(enrolment.Id);
    }
```

```
    public async Task<OperationResult> UpdateAsync(UpdateEnrolmentRequest request,
CancellationToken ct = default)
    {
        var enrolment = await db.Enrolments.FirstOrDefaultAsync(e => e.Id == request.Id,
ct);
        if (enrolment is null) return OperationResult.Failure("Enrolment not found.");

        if (!await db.SchoolClasses.AnyAsync(c => c.Id == request.SchoolClassId, ct))
            return OperationResult.Failure("Class not found.");

        if (!await db.EnrolmentStatuses.AnyAsync(s => s.Id == request.EnrolmentStatusId,
ct))
            return OperationResult.Failure("Enrolment status is not valid.");

        if (request.WithdrawnOn.HasValue && request.WithdrawnOn.Value < request.EnrolledOn)
            return OperationResult.Failure("Withdrawal date cannot be earlier than the
enrolment date.");

        if (await db.Enrolments.AnyAsync(e =>
                e.StudentId == enrolment.StudentId
                && e.SchoolClassId == request.SchoolClassId
                && e.Id != request.Id, ct))
            return OperationResult.Failure("Student is already enrolled in this class.");

        enrolment.SchoolClassId = request.SchoolClassId;
        enrolment.EnrolledOn = request.EnrolledOn;
        enrolment.WithdrawnOn = request.WithdrawnOn;
        enrolment.EnrolmentStatusId = request.EnrolmentStatusId;
        enrolment.Notes = request.Notes;

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> WithdrawAsync(Guid id, DateOnly withdrawnOn, string?
notes, CancellationToken ct = default)
    {
        var enrolment = await db.Enrolments.FirstOrDefaultAsync(e => e.Id == id, ct);
        if (enrolment is null) return OperationResult.Failure("Enrolment not found.");

        if (withdrawnOn < enrolment.EnrolledOn)
            return OperationResult.Failure("Withdrawal date cannot be earlier than the
enrolment date.");

        var withdrawnStatus = await db.EnrolmentStatuses
            .FirstOrDefaultAsync(s => s.Name == "Withdrawn", ct);

        enrolment.WithdrawnOn = withdrawnOn;
        if (withdrawnStatus is not null) enrolment.EnrolmentStatusId = withdrawnStatus.Id;
        if (!string.IsNullOrWhiteSpace(notes))
        {
            enrolment.Notes = string.IsNullOrWhiteSpace(enrolment.Notes)
                ? notes
```

```
                : enrolment.Notes + "\n" + notes;
        }

        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }

    public async Task<OperationResult> SoftDeleteAsync(Guid id, CancellationToken ct =
default)
    {
        var enrolment = await db.Enrolments.FirstOrDefaultAsync(e => e.Id == id, ct);
        if (enrolment is null) return OperationResult.Failure("Enrolment not found.");

        db.Enrolments.Remove(enrolment);
        await db.SaveChangesAsync(ct);
        return OperationResult.Success();
    }
}
```

Three details deserve a sentence each:

- CreateAsync rejects an enrolment whose pupil already has an open enrolment in the same session. This is the rule that prevents 'pupil is in Primary 1A AND Primary 1B in 2025/2026'.
- WithdrawAsync looks up 'Withdrawn' by name in the EnrolmentStatuses table. If a school has renamed that row, the service still sets WithdrawnOn — it just leaves the status at whatever it was. WithdrawnOn is the load-bearing column; the status is a label.
- UpdateAsync re-checks the (StudentId, SchoolClassId) uniqueness constraint with .Id != request.Id, mirroring the sprint 2 services' pattern of pre-checking unique indexes for friendly error messages.

6.4 LookupService — the seven new methods

LookupService picks up seven new methods. Four expose the new lookup tables (relationships, enrolment statuses, blood groups, marital statuses); one is a session-scoped class lookup; and two are search helpers for students and parents.

Listing — src/NaijaPrimeSchool.Infrastructure/Services/LookupService.cs

```
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;
using NaijaPrimeSchool.Application.Users;
using NaijaPrimeSchool.Application.Users.Dtos;
using NaijaPrimeSchool.Domain.Identity;
using NaijaPrimeSchool.Infrastructure.Persistence;

namespace NaijaPrimeSchool.Infrastructure.Services;

public class LookupService(
    ApplicationDbContext db,
    UserManager<ApplicationUser> userManager) : ILookupService
{
    public async Task<IReadOnlyList<LookupDto>> GetGendersAsync(CancellationToken ct =
default) =>
```

```
        await db.Genders
            .OrderBy(g => g.DisplayOrder)
            .Select(g => new LookupDto { Id = g.Id, Name = g.Name, Code = g.Code })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetTitlesAsync(CancellationToken ct =
default) =>
        await db.Titles
            .OrderBy(t => t.DisplayOrder)
            .Select(t => new LookupDto { Id = t.Id, Name = t.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<RoleDto>> GetRolesAsync(CancellationToken ct = default)
=>
        await db.Roles
            .OrderBy(r => r.Name)
            .Select(r => new RoleDto
            {
                Id = r.Id,
                Name = r.Name!,
                Description = r.Description,
                IsSystemRole = r.IsSystemRole,
            })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetTermTypesAsync(CancellationToken ct =
default) =>
        await db.TermTypes
            .OrderBy(t => t.DisplayOrder)
            .Select(t => new LookupDto { Id = t.Id, Name = t.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetClassLevelsAsync(CancellationToken ct =
default) =>
        await db.ClassLevels
            .OrderBy(l => l.DisplayOrder)
            .Select(l => new LookupDto { Id = l.Id, Name = l.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetWeekDaysAsync(CancellationToken ct =
default) =>
        await db.WeekDays
            .OrderBy(d => d.DisplayOrder)
            .Select(d => new LookupDto { Id = d.Id, Name = d.Name, Code = d.ShortName })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetTeachersAsync(CancellationToken ct =
default)
    {
        var teachers = await userManager.GetUsersInRoleAsync(Roles.Teacher);
        return teachers
            .Where(u => u.IsActive)
            .OrderBy(u => u.FirstName)
            .ThenBy(u => u.LastName)
```

```
            .Select(u => new LookupDto
            {
                Id = u.Id,
                Name = (u.FirstName + " " + u.LastName).Trim(),
                Code = u.UserName,
            })
            .ToList();
    }

    public async Task<IReadOnlyList<LookupDto>> GetRelationshipsAsync(CancellationToken ct
= default) =>
        await db.Relationships
            .OrderBy(r => r.DisplayOrder)
            .Select(r => new LookupDto { Id = r.Id, Name = r.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetEnrolmentStatusesAsync(CancellationToken
ct = default) =>
        await db.EnrolmentStatuses
            .OrderBy(s => s.DisplayOrder)
            .Select(s => new LookupDto { Id = s.Id, Name = s.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetBloodGroupsAsync(CancellationToken ct =
default) =>
        await db.BloodGroups
            .OrderBy(g => g.DisplayOrder)
            .Select(g => new LookupDto { Id = g.Id, Name = g.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetMaritalStatusesAsync(CancellationToken
ct = default) =>
        await db.MaritalStatuses
            .OrderBy(m => m.DisplayOrder)
            .Select(m => new LookupDto { Id = m.Id, Name = m.Name })
            .ToListAsync(ct);

    public async Task<IReadOnlyList<LookupDto>> GetClassesForSessionAsync(Guid? sessionId =
null, CancellationToken ct = default)
    {
        var query = db.SchoolClasses.AsQueryable();
        if (sessionId.HasValue)
            query = query.Where(c => c.SessionId == sessionId.Value);

        return await query
            .OrderBy(c => c.ClassLevel!.DisplayOrder)
            .ThenBy(c => c.Name)
            .Select(c => new LookupDto
            {
                Id = c.Id,
                Name = c.Name + " — " + c.Session!.Name,
                Code = c.Session!.Name,
            })
            .ToListAsync(ct);
```

```
    }

    public async Task<IReadOnlyList<LookupDto>> GetStudentsAsync(string? searchTerm = null,
CancellationToken ct = default)
    {
        var query = db.Students.AsQueryable();
        if (!string.IsNullOrWhiteSpace(searchTerm))
        {
            var term = searchTerm.Trim().ToLower();
            query = query.Where(s =>
                s.FirstName.ToLower().Contains(term) ||
                s.LastName.ToLower().Contains(term) ||
                s.AdmissionNumber.ToLower().Contains(term));
        }

        return await query
            .OrderBy(s => s.FirstName).ThenBy(s => s.LastName)
            .Take(50)
            .Select(s => new LookupDto
            {
                Id = s.Id,
                Name = (s.FirstName + " " + s.LastName).Trim(),
                Code = s.AdmissionNumber,
            })
            .ToListAsync(ct);
    }

    public async Task<IReadOnlyList<LookupDto>> GetParentsAsync(string? searchTerm = null,
CancellationToken ct = default)
    {
        var query = db.Parents.AsQueryable();
        if (!string.IsNullOrWhiteSpace(searchTerm))
        {
            var term = searchTerm.Trim().ToLower();
            query = query.Where(p =>
                p.FirstName.ToLower().Contains(term) ||
                p.LastName.ToLower().Contains(term) ||
                (p.PrimaryPhone != null && p.PrimaryPhone.Contains(term)) ||
                (p.Email != null && p.Email.ToLower().Contains(term)));
        }

        return await query
            .OrderBy(p => p.FirstName).ThenBy(p => p.LastName)
            .Take(50)
            .Select(p => new LookupDto
            {
                Id = p.Id,
                Name = (p.FirstName + " " + p.LastName).Trim(),
                Code = p.PrimaryPhone,
            })
            .ToListAsync(ct);
    }
}
```

GetStudentsAsync and GetParentsAsync take an optional search term and cap the result at 50 rows. The cap matters because these methods feed the AllowFiltering dropdown on the parent-link form — Radzen renders every option in the DOM, so a list of 5,000 pupils would crater the page.

6.5 DI registration

DependencyInjection.cs picks up three new lines.

Excerpt — DependencyInjection.cs (sprint 3 additions)

```
        services.AddScoped<IStudentService, StudentService>();
        services.AddScoped<IParentService, ParentService>();
        services.AddScoped<IEnrolmentService, EnrolmentService>();

        return services;
```

7. The EF Core migration

A single migration named StudentsAndParents adds eight new tables (Relationships, EnrolmentStatuses, BloodGroups, MaritalStatuses, Students, Parents, StudentParents, Enrolments) and the indexes that go with them. It was generated with:

```
dotnet ef migrations add StudentsAndParents \
  --project src/NaijaPrimeSchool.Infrastructure \
  --startup-project src/NaijaPrimeSchool.Web \
  --output-dir Persistence/Migrations
```

On a fresh checkout the migration runs at startup via DatabaseInitializer.MigrateAsync. The full Up() method is embedded below for reference.

Excerpt — Up() of 20260429211125_StudentsAndParents.cs

```
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "BloodGroups",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_BloodGroups", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "EnrolmentStatuses",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(40)", maxLength: 40,
nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
```

```
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_EnrolmentStatuses", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "MaritalStatuses",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(40)", maxLength: 40,
nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_MaritalStatuses", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "Relationships",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    Name = table.Column<string>(type: "nvarchar(40)", maxLength: 40,
nullable: false),
                    DisplayOrder = table.Column<int>(type: "int", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
```

```
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Relationships", x => x.Id);
                });

            migrationBuilder.CreateTable(
                name: "Students",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    AdmissionNumber = table.Column<string>(type: "nvarchar(30)", maxLength:
30, nullable: false),
                    AdmissionDate = table.Column<DateOnly>(type: "date", nullable: false),
                    FirstName = table.Column<string>(type: "nvarchar(80)", maxLength: 80,
nullable: false),
                    LastName = table.Column<string>(type: "nvarchar(80)", maxLength: 80,
nullable: false),
                    MiddleName = table.Column<string>(type: "nvarchar(80)", maxLength: 80,
nullable: true),
                    DateOfBirth = table.Column<DateOnly>(type: "date", nullable: false),
                    GenderId = table.Column<Guid>(type: "uniqueidentifier", nullable:
true),
                    BloodGroupId = table.Column<Guid>(type: "uniqueidentifier", nullable:
true),
                    StateOfOrigin = table.Column<string>(type: "nvarchar(80)", maxLength:
80, nullable: true),
                    ResidentialAddress = table.Column<string>(type: "nvarchar(300)",
maxLength: 300, nullable: true),
                    PhotoUrl = table.Column<string>(type: "nvarchar(500)", maxLength: 500,
nullable: true),
                    Allergies = table.Column<string>(type: "nvarchar(500)", maxLength: 500,
nullable: true),
                    MedicalNotes = table.Column<string>(type: "nvarchar(1000)", maxLength:
1000, nullable: true),
                    UserId = table.Column<Guid>(type: "uniqueidentifier", nullable: true),
                    IsActive = table.Column<bool>(type: "bit", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
```

```
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Students", x => x.Id);
                    table.ForeignKey(
                        name: "FK_Students_BloodGroups_BloodGroupId",
                        column: x => x.BloodGroupId,
                        principalTable: "BloodGroups",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_Students_Genders_GenderId",
                        column: x => x.GenderId,
                        principalTable: "Genders",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_Students_Users_UserId",
                        column: x => x.UserId,
                        principalTable: "Users",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                });

            migrationBuilder.CreateTable(
                name: "Parents",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    FirstName = table.Column<string>(type: "nvarchar(80)", maxLength: 80,
nullable: false),
                    LastName = table.Column<string>(type: "nvarchar(80)", maxLength: 80,
nullable: false),
                    MiddleName = table.Column<string>(type: "nvarchar(80)", maxLength: 80,
nullable: true),
                    TitleId = table.Column<Guid>(type: "uniqueidentifier", nullable: true),
                    GenderId = table.Column<Guid>(type: "uniqueidentifier", nullable:
true),
                    MaritalStatusId = table.Column<Guid>(type: "uniqueidentifier",
nullable: true),
                    PrimaryPhone = table.Column<string>(type: "nvarchar(30)", maxLength:
30, nullable: true),
                    AlternatePhone = table.Column<string>(type: "nvarchar(30)", maxLength:
30, nullable: true),
                    Email = table.Column<string>(type: "nvarchar(256)", maxLength: 256,
nullable: true),
                    ResidentialAddress = table.Column<string>(type: "nvarchar(300)",
maxLength: 300, nullable: true),
                    Occupation = table.Column<string>(type: "nvarchar(120)", maxLength:
```

```
120, nullable: true),
                    Employer = table.Column<string>(type: "nvarchar(120)", maxLength: 120,
nullable: true),
                    UserId = table.Column<Guid>(type: "uniqueidentifier", nullable: true),
                    IsActive = table.Column<bool>(type: "bit", nullable: false),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Parents", x => x.Id);
                    table.ForeignKey(
                        name: "FK_Parents_Genders_GenderId",
                        column: x => x.GenderId,
                        principalTable: "Genders",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_Parents_MaritalStatuses_MaritalStatusId",
                        column: x => x.MaritalStatusId,
                        principalTable: "MaritalStatuses",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_Parents_Titles_TitleId",
                        column: x => x.TitleId,
                        principalTable: "Titles",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                    table.ForeignKey(
                        name: "FK_Parents_Users_UserId",
                        column: x => x.UserId,
                        principalTable: "Users",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.SetNull);
                });

            migrationBuilder.CreateTable(
                name: "Enrolments",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    StudentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
```

```
                    SchoolClassId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    EnrolledOn = table.Column<DateOnly>(type: "date", nullable: false),
                    WithdrawnOn = table.Column<DateOnly>(type: "date", nullable: true),
                    EnrolmentStatusId = table.Column<Guid>(type: "uniqueidentifier",
nullable: false),
                    Notes = table.Column<string>(type: "nvarchar(500)", maxLength: 500,
nullable: true),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Enrolments", x => x.Id);
                    table.ForeignKey(
                        name: "FK_Enrolments_EnrolmentStatuses_EnrolmentStatusId",
                        column: x => x.EnrolmentStatusId,
                        principalTable: "EnrolmentStatuses",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_Enrolments_SchoolClasses_SchoolClassId",
                        column: x => x.SchoolClassId,
                        principalTable: "SchoolClasses",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_Enrolments_Students_StudentId",
                        column: x => x.StudentId,
                        principalTable: "Students",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Cascade);
                });

            migrationBuilder.CreateTable(
                name: "StudentParents",
                columns: table => new
                {
                    Id = table.Column<Guid>(type: "uniqueidentifier", nullable: false),
                    StudentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    ParentId = table.Column<Guid>(type: "uniqueidentifier", nullable:
false),
                    RelationshipId = table.Column<Guid>(type: "uniqueidentifier", nullable:
```

```
false),
                    IsPrimaryContact = table.Column<bool>(type: "bit", nullable: false),
                    CanPickUp = table.Column<bool>(type: "bit", nullable: false),
                    Notes = table.Column<string>(type: "nvarchar(300)", maxLength: 300,
nullable: true),
                    CreatedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: false),
                    CreatedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true),
                    ModifiedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    ModifiedBy = table.Column<string>(type: "nvarchar(100)", maxLength:
100, nullable: true),
                    IsDeleted = table.Column<bool>(type: "bit", nullable: false),
                    DeletedOn = table.Column<DateTimeOffset>(type: "datetimeoffset",
nullable: true),
                    DeletedBy = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
nullable: true)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_StudentParents", x => x.Id);
                    table.ForeignKey(
                        name: "FK_StudentParents_Parents_ParentId",
                        column: x => x.ParentId,
                        principalTable: "Parents",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Cascade);
                    table.ForeignKey(
                        name: "FK_StudentParents_Relationships_RelationshipId",
                        column: x => x.RelationshipId,
                        principalTable: "Relationships",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Restrict);
                    table.ForeignKey(
                        name: "FK_StudentParents_Students_StudentId",
                        column: x => x.StudentId,
                        principalTable: "Students",
                        principalColumn: "Id",
                        onDelete: ReferentialAction.Cascade);
                });

            migrationBuilder.CreateIndex(
                name: "IX_BloodGroups_Name",
                table: "BloodGroups",
                column: "Name",
                unique: true);

            migrationBuilder.CreateIndex(
                name: "IX_Enrolments_EnrolmentStatusId",
                table: "Enrolments",
                column: "EnrolmentStatusId");

            migrationBuilder.CreateIndex(
                name: "IX_Enrolments_IsDeleted",
```

```
            table: "Enrolments",
            column: "IsDeleted");

        migrationBuilder.CreateIndex(
            name: "IX_Enrolments_SchoolClassId",
            table: "Enrolments",
            column: "SchoolClassId");

        migrationBuilder.CreateIndex(
            name: "IX_Enrolments_StudentId_SchoolClassId",
            table: "Enrolments",
            columns: new[] { "StudentId", "SchoolClassId" },
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_EnrolmentStatuses_Name",
            table: "EnrolmentStatuses",
            column: "Name",
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_MaritalStatuses_Name",
            table: "MaritalStatuses",
            column: "Name",
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_Parents_GenderId",
            table: "Parents",
            column: "GenderId");

        migrationBuilder.CreateIndex(
            name: "IX_Parents_IsDeleted",
            table: "Parents",
            column: "IsDeleted");

        migrationBuilder.CreateIndex(
            name: "IX_Parents_MaritalStatusId",
            table: "Parents",
            column: "MaritalStatusId");

        migrationBuilder.CreateIndex(
            name: "IX_Parents_TitleId",
            table: "Parents",
            column: "TitleId");

        migrationBuilder.CreateIndex(
            name: "IX_Parents_UserId",
            table: "Parents",
            column: "UserId",
            unique: true,
            filter: "[UserId] IS NOT NULL");

        migrationBuilder.CreateIndex(
```

```
            name: "IX_Relationships_Name",
            table: "Relationships",
            column: "Name",
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_StudentParents_IsDeleted",
            table: "StudentParents",
            column: "IsDeleted");

        migrationBuilder.CreateIndex(
            name: "IX_StudentParents_ParentId",
            table: "StudentParents",
            column: "ParentId");

        migrationBuilder.CreateIndex(
            name: "IX_StudentParents_RelationshipId",
            table: "StudentParents",
            column: "RelationshipId");

        migrationBuilder.CreateIndex(
            name: "IX_StudentParents_StudentId_ParentId",
            table: "StudentParents",
            columns: new[] { "StudentId", "ParentId" },
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_Students_AdmissionNumber",
            table: "Students",
            column: "AdmissionNumber",
            unique: true);

        migrationBuilder.CreateIndex(
            name: "IX_Students_BloodGroupId",
            table: "Students",
            column: "BloodGroupId");

        migrationBuilder.CreateIndex(
            name: "IX_Students_GenderId",
            table: "Students",
            column: "GenderId");

        migrationBuilder.CreateIndex(
            name: "IX_Students_IsDeleted",
            table: "Students",
            column: "IsDeleted");

        migrationBuilder.CreateIndex(
            name: "IX_Students_UserId",
            table: "Students",
            column: "UserId",
            unique: true,
            filter: "[UserId] IS NOT NULL");
    }
```

```
        /// <inheritdoc />
        protected override void Down(MigrationBuilder migrationBuilder)
```

Notice the sequence: lookup tables first (Relationships, EnrolmentStatuses, BloodGroups, MaritalStatuses), then the two big entity tables (Students, Parents) which depend on those lookups, then the join tables (StudentParents, Enrolments) which depend on the entity tables. EF Core computes this ordering automatically from the foreign-key graph; we did not have to specify it.

7.1 Indexes created

The migration creates 16 indexes on the new tables:

- Unique on Relationships.Name, EnrolmentStatuses.Name, BloodGroups.Name, MaritalStatuses.Name (4 unique).
- Unique on Students.AdmissionNumber (1 unique).
- Filtered unique on Students.UserId WHERE [UserId] IS NOT NULL (1 unique).
- Filtered unique on Parents.UserId WHERE [UserId] IS NOT NULL (1 unique).
- Unique on StudentParents.(StudentId, ParentId) (1 unique).
- Unique on Enrolments.(StudentId, SchoolClassId) (1 unique).
- Plus IsDeleted indexes on every soft-deletable table (8).

7.2 The schema warning

The migration emits one warning at generation time:

```
  warn: Microsoft.EntityFrameworkCore.Model.Validation[10622]
        Entity 'ApplicationRole' has a global query filter
        defined and is the required end of a relationship with
        the entity 'ApplicationUserRole'. ...
```

This warning is unrelated to sprint 3. It is a long-standing interaction between Identity's IdentityUserRole and our soft-delete query filter on ApplicationRole. The runtime behaviour is fine because we never soft-delete a role from the UI — roles are seeded once and stay there. Sprint 6 (or whichever sprint adds role administration) will revisit this.

8. Seeding the family lookups

DatabaseInitializer picks up a new SeedFamilyLookupsAsync method that is invoked between SeedAcademicLookupsAsync and SeedRolesAsync. It seeds the four new lookup tables with sensible defaults a Nigerian primary school would want on day one.

Excerpt — SeedFamilyLookupsAsync

```
    private static async Task SeedFamilyLookupsAsync(ApplicationDbContext db,
CancellationToken ct)
    {
        if (!await db.Relationships.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] relationships =
            [
                "Father",
                "Mother",
                "Stepfather",
                "Stepmother",
                "Grandfather",
                "Grandmother",
                "Uncle",
                "Aunt",
                "Guardian",
                "Other",
            ];
            for (var i = 0; i < relationships.Length; i++)
            {
                db.Relationships.Add(new Relationship { Name = relationships[i],
DisplayOrder = i + 1 });
            }
        }

        if (!await db.EnrolmentStatuses.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] statuses =
            [
                "Active",
                "Suspended",
                "Withdrawn",
                "Transferred",
                "Graduated",
            ];
            for (var i = 0; i < statuses.Length; i++)
            {
                db.EnrolmentStatuses.Add(new EnrolmentStatus { Name = statuses[i],
DisplayOrder = i + 1 });
            }
        }

        if (!await db.BloodGroups.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] groups = ["A+", "A-", "B+", "B-", "AB+", "AB-", "O+", "O-",
"Unknown"];
            for (var i = 0; i < groups.Length; i++)
```

```
            {
                db.BloodGroups.Add(new BloodGroup { Name = groups[i], DisplayOrder = i +
1 });
            }
        }

        if (!await db.MaritalStatuses.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] statuses = ["Single", "Married", "Divorced", "Widowed", "Separated"];
            for (var i = 0; i < statuses.Length; i++)
            {
                db.MaritalStatuses.Add(new MaritalStatus { Name = statuses[i], DisplayOrder
= i + 1 });
            }
        }

        await db.SaveChangesAsync(ct);
    }

    private static async Task SeedAcademicLookupsAsync(ApplicationDbContext db,
CancellationToken ct)
    {
        if (!await db.TermTypes.IgnoreQueryFilters().AnyAsync(ct))
        {
            db.TermTypes.AddRange(
                new TermType { Name = "First Term", DisplayOrder = 1 },
                new TermType { Name = "Second Term", DisplayOrder = 2 },
                new TermType { Name = "Third Term", DisplayOrder = 3 });
        }

        if (!await db.ClassLevels.IgnoreQueryFilters().AnyAsync(ct))
        {
            string[] levels =
            [
                "Creche",
                "Pre-Nursery",
                "Nursery 1",
                "Nursery 2",
                "KG 1",
                "KG 2",
                "Primary 1",
                "Primary 2",
                "Primary 3",
                "Primary 4",
                "Primary 5",
                "Primary 6",
            ];
            for (var i = 0; i < levels.Length; i++)
            {
                db.ClassLevels.Add(new ClassLevel { Name = levels[i], DisplayOrder = i +
1 });
            }
        }
```

```
        if (!await db.WeekDays.IgnoreQueryFilters().AnyAsync(ct))
        {
            db.WeekDays.AddRange(
                new WeekDay { Name = "Monday",    ShortName = "Mon", DisplayOrder = 1 },
                new WeekDay { Name = "Tuesday",   ShortName = "Tue", DisplayOrder = 2 },
                new WeekDay { Name = "Wednesday", ShortName = "Wed", DisplayOrder = 3 },
                new WeekDay { Name = "Thursday",  ShortName = "Thu", DisplayOrder = 4 },
                new WeekDay { Name = "Friday",    ShortName = "Fri", DisplayOrder = 5 });
        }

        if (!await db.TimetablePeriods.IgnoreQueryFilters().AnyAsync(ct))
        {
            (string Name, int Hour, int Minute, int DurationMinutes, int Order, bool
IsBreak)[] periods =
            [
                ("Period 1",      8, 0,  40, 1, false),
                ("Period 2",      8, 40, 40, 2, false),
                ("Period 3",      9, 20, 40, 3, false),
                ("Short Break",  10, 0,  20, 4, true),
                ("Period 4",     10, 20, 40, 5, false),
                ("Period 5",     11, 0,  40, 6, false),
                ("Lunch",        11, 40, 40, 7, true),
                ("Period 6",     12, 20, 40, 8, false),
                ("Period 7",     13, 0,  40, 9, false),
            ];
            foreach (var p in periods)
            {
                var start = new TimeOnly(p.Hour, p.Minute);
                db.TimetablePeriods.Add(new TimetablePeriod
                {
                    Name = p.Name,
                    StartTime = start,
                    EndTime = start.AddMinutes(p.DurationMinutes),
                    DisplayOrder = p.Order,
                    IsBreak = p.IsBreak,
                });
            }
        }

        await db.SaveChangesAsync(ct);
    }

    private static async Task SeedLookupsAsync(ApplicationDbContext db, CancellationToken
ct)
```

What gets seeded:

- Relationships — Father, Mother, Stepfather, Stepmother, Grandfather, Grandmother, Uncle, Aunt, Guardian, Other (10 rows).
- EnrolmentStatuses — Active, Suspended, Withdrawn, Transferred, Graduated (5 rows). Active has DisplayOrder = 1, which is what CreateEnrolmentRequest defaults to when no status is specified.
- BloodGroups — A+, A-, B+, B-, AB+, AB-, O+, O-, Unknown (9 rows).

- MaritalStatuses — Single, Married, Divorced, Widowed, Separated (5 rows).

The seeder uses the same .IgnoreQueryFilters().AnyAsync() guard as the sprint 1 and sprint 2 seeders — it only inserts if the table is empty (counting soft-deleted rows), so running the app a second time after a soft delete does not resurrect the missing row.

9. The Razor pages

Six new pages land in src/NaijaPrimeSchool.Web/Components/Pages/Family/. The pattern mirrors sprint 2: a list page, a create page, an edit page, with the addition of an enrolment-overview page that does not have its own create/edit because enrolment is created and managed from within the student profile.

9.1 Page roster

```
src/NaijaPrimeSchool.Web/Components/Pages/Family/
├── Students.razor        ← /students       (list + filters)
├── CreateStudent.razor   ← /students/new
├── EditStudent.razor     ← /students/{id}  (Profile / Parents / Enrolments tabs)
├── Parents.razor         ← /parents        (list + search)
├── CreateParent.razor    ← /parents/new
├── EditParent.razor      ← /parents/{id}   (Profile / Linked pupils tabs)
└── Enrolments.razor      ← /enrolments     (overview + filters)
```

Every page is gated to SuperAdmin + HeadTeacher. Teachers, Bursars, Storekeepers, Parents and Students do not see the Family panel and cannot navigate to the routes — the route itself returns 403.

9.2 Students.razor — the directory

Students.razor is the front door. A search box, three dropdowns (session, class, status), and a paged grid. The grid renders the pupil's name and admission number in column 1, their current class and session in column 2 (or 'Not enrolled' if they have no open enrolment), the primary contact in column 3, DOB in column 4, status badge in column 5, and three action buttons in column 6.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Family/Students.razor

```
@page "/students"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IStudentService StudentService
@inject ILookupService LookupService
@inject ISessionService SessionService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Students · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Students</h1>
        <p>Pupils enrolled across the school. Add new pupils, manage profiles, and review
parent linkage.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Add new student" Icon="person_add"
                      ButtonStyle="ButtonStyle.Primary" Click="@(() =>
Nav.NavigateTo("/students/new"))" />
    </div>
</div>
```

```
<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Search</label>
            <RadzenTextBox @bind-Value="searchTerm" Placeholder="Name or admission number"
                           Change="@(_ => LoadAsync())" Style="min-width: 260px;" />
        </div>
        <div class="nps-field">
            <label>Session</label>
            <RadzenDropDown TValue="Guid?" Data="@sessions" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="sessionId" AllowClear="true" Placeholder="All
sessions"
                            Change="@(_ => SessionChangedAsync())" Style="min-width:
200px;" />
        </div>
        <div class="nps-field">
            <label>Class</label>
            <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="classId" AllowClear="true" Placeholder="All
classes"
                            Change="@(_ => LoadAsync())" Style="min-width: 240px;" />
        </div>
        <div class="nps-field">
            <label>Status</label>
            <RadzenDropDown TValue="bool?" @bind-Value="isActive"
                            Data="@statusOptions" TextProperty="Text" ValueProperty="Value"
                            AllowClear="true" Placeholder="All"
                            Change="@(_ => LoadAsync())" Style="min-width: 160px;" />
        </div>
        <RadzenButton Text="Refresh" Icon="refresh" Variant="Variant.Flat"
                      ButtonStyle="ButtonStyle.Light" Click="@LoadAsync" />
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid @ref="grid" TItem="StudentDto" Data="@students" Count="@total"
                    IsLoading="@loading" AllowPaging="true" AllowSorting="true"
                    PageSize="20" PageSizeOptions="@(new[] {10, 20, 50})"
                    LoadData="@OnLoadData" EmptyText="No students match your filters.">
        <Columns>
            <RadzenDataGridColumn TItem="StudentDto" Title="Pupil" Property="FullName">
                <Template Context="s">
                    <div style="display:flex; flex-direction:column;">
                        <strong>@s.FullName</strong>
                        <small style="color:
var(--nps-ink-500);">@s.AdmissionNumber</small>
                    </div>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StudentDto" Title="Current class"
Property="CurrentClassName">
                <Template Context="s">
```

```
                    @if (string.IsNullOrEmpty(s.CurrentClassName))
                    {
                        <span style="color: var(--nps-ink-500); font-style: italic;">Not
enrolled</span>
                    }
                    else
                    {
                        <strong>@s.CurrentClassName</strong>
                        <br />
                        <small style="color:
var(--nps-ink-500);">@s.CurrentSessionName</small>
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StudentDto" Title="Primary contact"
Sortable="false">
                <Template Context="s">
                    @if (string.IsNullOrEmpty(s.PrimaryContactName))
                    {
                        <RadzenBadge Text="No parent linked"
BadgeStyle="BadgeStyle.Warning" Variant="Variant.Flat" />
                    }
                    else
                    {
                        <div style="display:flex; flex-direction:column;">
                            <strong>@s.PrimaryContactName</strong>
                            <small style="color: var(--nps-
ink-500);">@s.PrimaryContactPhone</small>
                        </div>
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StudentDto" Title="DOB" Property="DateOfBirth"
                                  FormatString="{0:d MMM yyyy}" Width="140px" />
            <RadzenDataGridColumn TItem="StudentDto" Title="Status" Width="120px"
Property="IsActive">
                <Template Context="s">
                    @if (s.IsActive)
                    {
                        <RadzenBadge Text="Active" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                    }
                    else
                    {
                        <RadzenBadge Text="Inactive" BadgeStyle="BadgeStyle.Secondary"
Variant="Variant.Flat" />
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="StudentDto" Title="Actions" Sortable="false"
Filterable="false"
                                  TextAlign="TextAlign.Right" Width="220px">
                <Template Context="s">
                    <div class="nps-inline-actions">
                        <RadzenButton Icon="edit" Size="ButtonSize.Small"
```

```
Variant="Variant.Flat"
                                        ButtonStyle="ButtonStyle.Light" title="Edit"
                                        Click="@(() =>
Nav.NavigateTo($"/students/{s.Id}"))" />
                        <RadzenButton Icon="family_restroom" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                        ButtonStyle="ButtonStyle.Info" title="Manage parents"
                                        Click="@(() => Nav.NavigateTo($"/students/{s.Id}
#parents"))" />
                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                        ButtonStyle="ButtonStyle.Danger" title="Delete"
                                        Click="@(() => DeleteAsync(s))" />
                    </div>
                </Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
</RadzenCard>

@code {
    private RadzenDataGrid<StudentDto>? grid;
    private List<StudentDto> students = [];
    private int total;
    private bool loading;

    private string? searchTerm;
    private Guid? sessionId;
    private Guid? classId;
    private bool? isActive;

    private IReadOnlyList<SessionDto> sessions = [];
    private IReadOnlyList<LookupDto> classes = [];

    private record StatusOption(string Text, bool? Value);
    private readonly List<StatusOption> statusOptions =
    [
        new StatusOption("Active", true),
        new StatusOption("Inactive", false),
    ];

    private int skip;
    private int take = 20;
    private string? orderBy;

    protected override async Task OnInitializedAsync()
    {
        sessions = await SessionService.ListAsync();
        sessionId = sessions.FirstOrDefault(s => s.IsCurrent)?.Id;
        classes = await LookupService.GetClassesForSessionAsync(sessionId);
    }

    private async Task SessionChangedAsync()
    {
```

```
        classId = null;
        classes = await LookupService.GetClassesForSessionAsync(sessionId);
        await LoadAsync();
    }

    private async Task OnLoadData(LoadDataArgs args)
    {
        skip = args.Skip ?? 0;
        take = args.Top ?? 20;
        orderBy = args.OrderBy;
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            var result = await StudentService.ListAsync(new StudentListFilter
            {
                SearchTerm = searchTerm,
                SessionId = sessionId,
                ClassId = classId,
                IsActive = isActive,
                Skip = skip,
                Take = take,
                OrderBy = orderBy,
            });
            students = result.Items;
            total = result.TotalCount;
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }

    private async Task DeleteAsync(StudentDto s)
    {
        var confirm = await Dialog.Confirm(
            $"Delete student '{s.FullName}' ({s.AdmissionNumber})? Active enrolments must
be withdrawn first.",
            "Delete student",
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await StudentService.SoftDeleteAsync(s.Id);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Deleted", $"'{s.FullName}'
removed.");
            await LoadAsync();
        }
```

```
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }
}
```

The session dropdown defaults to the current session (looked up via SessionService.GetCurrentAsync), and the class dropdown is rebuilt whenever the session changes via LookupService.GetClassesForSessionAsync. The grid's LoadData event drives paging and sorting through the LINQ filter on the server.

9.3 CreateStudent.razor — the create form

CreateStudent.razor is a single long inline form, organised into three labelled sections: Identity, Health, and Initial enrolment. The Initial enrolment section is optional — if the admin picks a class, the service creates an enrolment row alongside the student in the same SaveChanges; if they skip it, the pupil is created without any enrolment and the admin can enrol them later from the EditStudent page.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Family/CreateStudent.razor

```
@page "/students/new"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IStudentService StudentService
@inject ILookupService LookupService
@inject ISessionService SessionService
@inject NavigationManager Nav
@inject NotificationService Notification

<PageTitle>New student · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>New student</h1>
        <p>Capture a pupil's profile and (optionally) enrol them straight into a class.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Back to students" Icon="arrow_back" Variant="Variant.Flat"
                      ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/students"))" />
    </div>
</div>

<RadzenCard class="nps-card">
    <RadzenTemplateForm TItem="StudentFormModel" Data="@form" Submit="@SaveAsync">
        <DataAnnotationsValidator />

        <h3 style="margin: 0 0 1rem;">Identity</h3>
        <div class="nps-form-grid">
            <div class="nps-field">
```

```
                <label>Admission number *</label>
                <RadzenTextBox @bind-Value="form.AdmissionNumber" Placeholder="e.g.
NPS/2026/0001" />
                <ValidationMessage For="() => form.AdmissionNumber" />
            </div>
            <div class="nps-field">
                <label>Admission date *</label>
                <RadzenDatePicker TValue="DateTime?" @bind-Value="form.AdmissionDate"
                                  DateFormat="d MMM yyyy" ShowTime="false" />
            </div>
            <div class="nps-field">
                <label>First name *</label>
                <RadzenTextBox @bind-Value="form.FirstName" />
                <ValidationMessage For="() => form.FirstName" />
            </div>
            <div class="nps-field">
                <label>Last name *</label>
                <RadzenTextBox @bind-Value="form.LastName" />
                <ValidationMessage For="() => form.LastName" />
            </div>
            <div class="nps-field">
                <label>Middle name</label>
                <RadzenTextBox @bind-Value="form.MiddleName" />
            </div>
            <div class="nps-field">
                <label>Date of birth *</label>
                <RadzenDatePicker TValue="DateTime?" @bind-Value="form.DateOfBirth"
                                  DateFormat="d MMM yyyy" ShowTime="false" />
            </div>
            <div class="nps-field">
                <label>Gender</label>
                <RadzenDropDown TValue="Guid?" Data="@genders" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="form.GenderId" AllowClear="true"
Placeholder="Pick one" />
            </div>
            <div class="nps-field">
                <label>Blood group</label>
                <RadzenDropDown TValue="Guid?" Data="@bloodGroups" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="form.BloodGroupId" AllowClear="true"
Placeholder="Pick one" />
            </div>
            <div class="nps-field">
                <label>State of origin</label>
                <RadzenTextBox @bind-Value="form.StateOfOrigin" Placeholder="e.g. Ogun" />
            </div>
            <div class="nps-field nps-span-2">
                <label>Residential address</label>
                <RadzenTextArea @bind-Value="form.ResidentialAddress" Rows="2" />
            </div>
        </div>

        <h3 style="margin: 1.5rem 0 1rem;">Health</h3>
        <div class="nps-form-grid">
```

```
            <div class="nps-field nps-span-2">
                <label>Allergies</label>
                <RadzenTextArea @bind-Value="form.Allergies" Rows="2"
                                Placeholder="Known allergies, if any (medication, food,
environmental)" />
            </div>
            <div class="nps-field nps-span-2">
                <label>Medical notes</label>
                <RadzenTextArea @bind-Value="form.MedicalNotes" Rows="3"
                                Placeholder="Conditions or medication that staff should be
aware of" />
            </div>
        </div>

        <h3 style="margin: 1.5rem 0 1rem;">Initial enrolment (optional)</h3>
        <div class="nps-form-grid">
            <div class="nps-field nps-span-2">
                <label>Class</label>
                <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="form.InitialClassId" AllowClear="true"
                                Placeholder="Pick a class to enrol straight away (or skip)"
/>
                <small style="color: var(--nps-ink-500); margin-top: 0.25rem;">
                    The pupil's admission date will be used as the enrolment date.
                </small>
            </div>
        </div>

        <div class="nps-form-actions">
            <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                          ButtonStyle="ButtonStyle.Light" Variant="Variant.Flat"
                          Click="@(() => Nav.NavigateTo("/students"))" />
            <RadzenButton Text="@(saving ? "Saving..." : "Save student")"
                          Icon="save" ButtonType="ButtonType.Submit"
                          ButtonStyle="ButtonStyle.Primary" Disabled="@saving" />
        </div>
    </RadzenTemplateForm>
</RadzenCard>

@code {
    private StudentFormModel form = new();
    private bool saving;

    private IReadOnlyList<LookupDto> genders = [];
    private IReadOnlyList<LookupDto> bloodGroups = [];
    private IReadOnlyList<LookupDto> classes = [];

    private class StudentFormModel
    {
        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(30)]
        public string AdmissionNumber { get; set; } = string.Empty;
```

```
        public DateTime? AdmissionDate { get; set; } = DateTime.Today;

        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(80)]
        public string FirstName { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(80)]
        public string LastName { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.StringLength(80)]
        public string? MiddleName { get; set; }

        public DateTime? DateOfBirth { get; set; }
        public Guid? GenderId { get; set; }
        public Guid? BloodGroupId { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(80)]
        public string? StateOfOrigin { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(300)]
        public string? ResidentialAddress { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(500)]
        public string? Allergies { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(1000)]
        public string? MedicalNotes { get; set; }

        public Guid? InitialClassId { get; set; }
    }

    protected override async Task OnInitializedAsync()
    {
        genders = await LookupService.GetGendersAsync();
        bloodGroups = await LookupService.GetBloodGroupsAsync();
        var current = await SessionService.GetCurrentAsync();
        classes = await LookupService.GetClassesForSessionAsync(current?.Id);
    }

    private async Task SaveAsync()
    {
        if (form.AdmissionDate is null || form.DateOfBirth is null)
        {
            Notification.Notify(NotificationSeverity.Warning, "Dates required",
                "Pick an admission date and a date of birth.");
            return;
        }

        saving = true;
        try
        {
            var result = await StudentService.CreateAsync(new CreateStudentRequest
            {
```

```
                AdmissionNumber = form.AdmissionNumber,
                AdmissionDate = DateOnly.FromDateTime(form.AdmissionDate.Value),
                FirstName = form.FirstName,
                LastName = form.LastName,
                MiddleName = form.MiddleName,
                DateOfBirth = DateOnly.FromDateTime(form.DateOfBirth.Value),
                GenderId = form.GenderId,
                BloodGroupId = form.BloodGroupId,
                StateOfOrigin = form.StateOfOrigin,
                ResidentialAddress = form.ResidentialAddress,
                Allergies = form.Allergies,
                MedicalNotes = form.MedicalNotes,
                InitialClassId = form.InitialClassId,
            });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved",
                    $"Student '{form.FirstName} {form.LastName}' created.");
                Nav.NavigateTo($"/students/{result.Data}");
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
            }
        }
        finally
        {
            saving = false;
        }
    }
}
```

The form is RadzenTemplateForm with DataAnnotationsValidator, so every required-field message and length-constraint message from the DTOs renders inline next to the offending field. On success, Nav.NavigateTo($"/students/{result.Data}") drops the admin straight onto the new pupil's edit page so they can immediately link parents.

9.4 EditStudent.razor — the heart of the sprint

EditStudent.razor is the most substantial page in the sprint. It uses Radzen tabs to present three views of the same pupil:

- Profile — every demographic and medical field, editable.
- Parents — the list of linked parents, with inline link/edit/unlink workflows.
- Enrolment history — the full chronology with actions for withdraw and delete.

There are also two top-right buttons that flip the pupil's active status (Activate / Deactivate). The whole page is rendered server-side (InteractiveServer) so every form submission is a SignalR message rather than

a client-side WebAssembly round-trip — the data is sensitive enough that we want every keystroke gated by the cookie auth.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Family/EditStudent.razor

```
@page "/students/{Id:guid}"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IStudentService StudentService
@inject IEnrolmentService EnrolmentService
@inject ILookupService LookupService
@inject ISessionService SessionService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Edit student · Naija Prime School</PageTitle>

@if (loadingProfile)
{
    <p>Loading student...</p>
}
else if (student is null)
{
    <RadzenAlert Severity="AlertSeverity.Warning">Student not found.</RadzenAlert>
}
else
{
    <div class="nps-page-header">
        <div>
            <h1>@student.FullName</h1>
            <p>Admission number <strong>@student.AdmissionNumber</strong> &middot;
               admitted @student.AdmissionDate.ToString("d MMM yyyy")</p>
        </div>
        <div class="nps-page-actions">
            @if (student.IsActive)
            {
                <RadzenButton Text="Deactivate" Icon="block" Variant="Variant.Flat"
                              ButtonStyle="ButtonStyle.Warning" Click="@(() =>
SetActiveAsync(false))" />
            }
            else
            {
                <RadzenButton Text="Activate" Icon="check_circle" Variant="Variant.Flat"
                              ButtonStyle="ButtonStyle.Success" Click="@(() =>
SetActiveAsync(true))" />
            }
            <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
                          ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/students"))" />
        </div>
    </div>

    <RadzenTabs @bind-SelectedIndex="selectedTab">
```

```
        <Tabs>
            <RadzenTabsItem Text="Profile">
                <RadzenCard class="nps-card">
                    <RadzenTemplateForm TItem="StudentFormModel" Data="@form"
Submit="@SaveAsync">
                        <DataAnnotationsValidator />
                        <div class="nps-form-grid">
                            <div class="nps-field">
                                <label>Admission number *</label>
                                <RadzenTextBox @bind-Value="form.AdmissionNumber" />
                                <ValidationMessage For="() => form.AdmissionNumber" />
                            </div>
                            <div class="nps-field">
                                <label>Admission date *</label>
                                <RadzenDatePicker TValue="DateTime?" @bind-
Value="form.AdmissionDate"
                                                  DateFormat="d MMM yyyy"
ShowTime="false" />
                            </div>
                            <div class="nps-field">
                                <label>First name *</label>
                                <RadzenTextBox @bind-Value="form.FirstName" />
                            </div>
                            <div class="nps-field">
                                <label>Last name *</label>
                                <RadzenTextBox @bind-Value="form.LastName" />
                            </div>
                            <div class="nps-field">
                                <label>Middle name</label>
                                <RadzenTextBox @bind-Value="form.MiddleName" />
                            </div>
                            <div class="nps-field">
                                <label>Date of birth *</label>
                                <RadzenDatePicker TValue="DateTime?" @bind-
Value="form.DateOfBirth"
                                                  DateFormat="d MMM yyyy"
ShowTime="false" />
                            </div>
                            <div class="nps-field">
                                <label>Gender</label>
                                <RadzenDropDown TValue="Guid?" Data="@genders"
TextProperty="Name" ValueProperty="Id"
                                                @bind-Value="form.GenderId"
AllowClear="true" />
                            </div>
                            <div class="nps-field">
                                <label>Blood group</label>
                                <RadzenDropDown TValue="Guid?" Data="@bloodGroups"
TextProperty="Name" ValueProperty="Id"
                                                @bind-Value="form.BloodGroupId"
AllowClear="true" />
                            </div>
                            <div class="nps-field">
                                <label>State of origin</label>
                                <RadzenTextBox @bind-Value="form.StateOfOrigin" />
```

```
                            </div>
                            <div class="nps-field nps-span-2">
                                <label>Residential address</label>
                                <RadzenTextArea @bind-Value="form.ResidentialAddress"
Rows="2" />
                            </div>
                            <div class="nps-field nps-span-2">
                                <label>Allergies</label>
                                <RadzenTextArea @bind-Value="form.Allergies" Rows="2" />
                            </div>
                            <div class="nps-field nps-span-2">
                                <label>Medical notes</label>
                                <RadzenTextArea @bind-Value="form.MedicalNotes" Rows="3" />
                            </div>
                        </div>
                        <div class="nps-form-actions">
                            <RadzenButton Text="@(saving ? "Saving..." : "Save changes")"
                                          Icon="save" ButtonType="ButtonType.Submit"
                                          ButtonStyle="ButtonStyle.Primary"
Disabled="@saving" />
                        </div>
                    </RadzenTemplateForm>
                </RadzenCard>
            </RadzenTabsItem>

            <RadzenTabsItem Text="@($"Parents ({parentLinks.Count})")">
                <RadzenCard class="nps-card">
                    <div style="display:flex; justify-content: space-between; align-items:
center; margin-bottom: 1rem;">
                        <h3 style="margin:0;">Linked parents and guardians</h3>
                        <RadzenButton Text="Link a parent" Icon="add"
ButtonStyle="ButtonStyle.Primary"
                                      Click="@OpenLinkForm" />
                    </div>

                    @if (parentLinks.Count == 0)
                    {
                        <p style="color: var(--nps-ink-500);">
                            No parents are linked yet. Click <strong>Link a parent</strong>
to choose from
                            the parents directory, or add a new parent first under Family →
Parents.
                        </p>
                    }
                    else
                    {
                        <RadzenDataGrid TItem="StudentParentDto" Data="@parentLinks"
AllowPaging="false"
                                        EmptyText="No parents linked.">
                            <Columns>
                                <RadzenDataGridColumn TItem="StudentParentDto"
Title="Parent" Property="ParentName">
                                    <Template Context="l">
                                        <strong>@l.ParentName</strong>
                                        @if (l.IsPrimaryContact)
```

```
                                        {
                                            <RadzenBadge Text="Primary"
BadgeStyle="BadgeStyle.Success"
                                                         Variant="Variant.Flat"
Style="margin-left:.5rem;" />
                                        }
                                        @if (!l.CanPickUp)
                                        {
                                            <RadzenBadge Text="Cannot pick up"
BadgeStyle="BadgeStyle.Warning"
                                                         Variant="Variant.Flat"
Style="margin-left:.5rem;" />
                                        }
                                        <br />
                                        <small style="color: var(--nps-ink-500);">
                                            @l.ParentPhone
@(string.IsNullOrEmpty(l.ParentEmail) ? "" : " · " + l.ParentEmail)
                                        </small>
                                    </Template>
                                </RadzenDataGridColumn>
                                <RadzenDataGridColumn TItem="StudentParentDto"
Title="Relationship" Property="RelationshipName" Width="160px" />
                                <RadzenDataGridColumn TItem="StudentParentDto"
Title="Notes" Property="Notes" />
                                <RadzenDataGridColumn TItem="StudentParentDto"
Title="Actions"
                                                      Width="160px" Sortable="false"
TextAlign="TextAlign.Right">
                                    <Template Context="l">
                                        <div class="nps-inline-actions">
                                            <RadzenButton Icon="edit"
Size="ButtonSize.Small" Variant="Variant.Flat"
                                                          ButtonStyle="ButtonStyle.Light"
                                                          Click="@(() =>
OpenLinkEditForm(l))" title="Edit link" />
                                            <RadzenButton Icon="link_off"
Size="ButtonSize.Small" Variant="Variant.Flat"
                                                          ButtonStyle="ButtonStyle.Danger"
                                                          Click="@(() => UnlinkAsync(l))"
title="Unlink" />
                                        </div>
                                    </Template>
                                </RadzenDataGridColumn>
                            </Columns>
                        </RadzenDataGrid>
                    }

                    @if (showLinkForm)
                    {
                        <RadzenCard class="nps-card" Style="margin-top:1rem; background:
var(--nps-green-200);">
                            <h3 style="margin:0 0 1rem;">@(linkEditingId.HasValue ? "Edit
link" : "Link a parent")</h3>
                            <div class="nps-form-grid">
                                @if (!linkEditingId.HasValue)
```

```
                                {
                                    <div class="nps-field nps-span-2">
                                        <label>Parent *</label>
                                        <RadzenDropDown TValue="Guid?" Data="@parents"
TextProperty="Name" ValueProperty="Id"
                                                        @bind-Value="linkForm.ParentId"
Placeholder="Search and pick a parent"
                                                        AllowFiltering="true"
FilterCaseSensitivity="FilterCaseSensitivity.CaseInsensitive" />
                                    </div>
                                }
                                <div class="nps-field">
                                    <label>Relationship *</label>
                                    <RadzenDropDown TValue="Guid?" Data="@relationships"
TextProperty="Name" ValueProperty="Id"
                                                    @bind-Value="linkForm.RelationshipId"
Placeholder="e.g. Father" />
                                </div>
                                <div class="nps-field">
                                    <label>Primary contact</label>
                                    <RadzenSwitch @bind-
Value="linkForm.IsPrimaryContact" />
                                </div>
                                <div class="nps-field">
                                    <label>Authorised to collect</label>
                                    <RadzenSwitch @bind-Value="linkForm.CanPickUp" />
                                </div>
                                <div class="nps-field nps-span-2">
                                    <label>Notes</label>
                                    <RadzenTextArea @bind-Value="linkForm.Notes"
Rows="2" />
                                </div>
                            </div>
                            <div class="nps-form-actions">
                                <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                                              ButtonStyle="ButtonStyle.Light"
Variant="Variant.Flat"
                                              Click="@(() => showLinkForm = false)" />
                                <RadzenButton Text="@(linkSaving ? "Saving..." : "Save
link")"
                                              Icon="save" ButtonStyle="ButtonStyle.Primary"
                                              Disabled="@linkSaving" Click="@SaveLinkAsync"
/>
                            </div>
                        </RadzenCard>
                    }
                </RadzenCard>
            </RadzenTabsItem>

            <RadzenTabsItem Text="@($"Enrolment history ({enrolments.Count})")">
                <RadzenCard class="nps-card">
                    <div style="display:flex; justify-content: space-between; align-items:
center; margin-bottom: 1rem;">
                        <h3 style="margin:0;">Enrolments</h3>
                        <RadzenButton Text="New enrolment" Icon="add"
```

```
ButtonStyle="ButtonStyle.Primary"
                                    Click="@OpenEnrolmentForm" />
                    </div>
                    <RadzenDataGrid TItem="EnrolmentDto" Data="@enrolments"
AllowPaging="false"
                                    EmptyText="No enrolment records yet.">
                        <Columns>
                            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Class">
                                <Template Context="e">
                                    <strong>@e.SchoolClassName</strong>
                                    <br />
                                    <small style="color: var(--nps-
ink-500);">@e.SessionName</small>
                                </Template>
                            </RadzenDataGridColumn>
                            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Enrolled"
Property="EnrolledOn"
                                                  FormatString="{0:d MMM yyyy}"
Width="140px" />
                            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Withdrawn"
Property="WithdrawnOn"
                                                  FormatString="{0:d MMM yyyy}"
Width="140px">
                                <Template Context="e">
                                    @(e.WithdrawnOn.HasValue ?
e.WithdrawnOn.Value.ToString("d MMM yyyy") : "—")
                                </Template>
                            </RadzenDataGridColumn>
                            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Status"
Width="140px">
                                <Template Context="e">
                                    <RadzenBadge Text="@e.EnrolmentStatusName"
                                                 BadgeStyle="@(e.WithdrawnOn.HasValue ?
BadgeStyle.Secondary : BadgeStyle.Success)"
                                                 Variant="Variant.Flat" />
                                </Template>
                            </RadzenDataGridColumn>
                            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Actions"
                                                   Width="160px" Sortable="false"
TextAlign="TextAlign.Right">
                                <Template Context="e">
                                    <div class="nps-inline-actions">
                                        @if (!e.WithdrawnOn.HasValue)
                                        {
                                            <RadzenButton Icon="logout"
Size="ButtonSize.Small" Variant="Variant.Flat"
                                                          ButtonStyle="ButtonStyle.Warning"
                                                          Click="@(() => WithdrawAsync(e))"
title="Withdraw" />
                                        }
                                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                                      ButtonStyle="ButtonStyle.Danger"
                                                      Click="@(() =>
DeleteEnrolmentAsync(e))" title="Delete" />
```

```
                                    </div>
                                </Template>
                            </RadzenDataGridColumn>
                        </Columns>
                    </RadzenDataGrid>

                    @if (showEnrolmentForm)
                    {
                        <RadzenCard class="nps-card" Style="margin-top:1rem; background:
var(--nps-green-200);">
                            <h3 style="margin:0 0 1rem;">New enrolment</h3>
                            <div class="nps-form-grid">
                                <div class="nps-field nps-span-2">
                                    <label>Class *</label>
                                    <RadzenDropDown TValue="Guid?" Data="@allClasses"
TextProperty="Name" ValueProperty="Id"
                                                    @bind-Value="enrolmentForm.ClassId"
                                                    Placeholder="Pick a class"
                                                    AllowFiltering="true" />
                                </div>
                                <div class="nps-field">
                                    <label>Enrolment date *</label>
                                    <RadzenDatePicker TValue="DateTime?" @bind-
Value="enrolmentForm.EnrolledOn"
                                                      DateFormat="d MMM yyyy"
ShowTime="false" />
                                </div>
                                <div class="nps-field nps-span-2">
                                    <label>Notes</label>
                                    <RadzenTextArea @bind-Value="enrolmentForm.Notes"
Rows="2" />
                                </div>
                            </div>
                            <div class="nps-form-actions">
                                <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                                              ButtonStyle="ButtonStyle.Light"
Variant="Variant.Flat"
                                              Click="@(() => showEnrolmentForm = false)" />
                                <RadzenButton Text="@(enrolmentSaving ? "Saving..." :
"Enrol")"
                                              Icon="save" ButtonStyle="ButtonStyle.Primary"
                                              Disabled="@enrolmentSaving"
Click="@SaveEnrolmentAsync" />
                            </div>
                        </RadzenCard>
                    }
                </RadzenCard>
            </RadzenTabsItem>
        </Tabs>
    </RadzenTabs>
}

@code {
    [Parameter] public Guid Id { get; set; }
```

```
    private bool loadingProfile = true;
    private bool saving;
    private int selectedTab;

    private StudentDto? student;
    private StudentFormModel form = new();

    private IReadOnlyList<LookupDto> genders = [];
    private IReadOnlyList<LookupDto> bloodGroups = [];
    private IReadOnlyList<LookupDto> relationships = [];
    private IReadOnlyList<LookupDto> parents = [];
    private IReadOnlyList<LookupDto> allClasses = [];

    private IReadOnlyList<StudentParentDto> parentLinks = [];
    private IReadOnlyList<EnrolmentDto> enrolments = [];

    private bool showLinkForm;
    private bool linkSaving;
    private Guid? linkEditingId;
    private LinkFormModel linkForm = new();

    private bool showEnrolmentForm;
    private bool enrolmentSaving;
    private EnrolmentFormModel enrolmentForm = new();

    private class StudentFormModel
    {
        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(30)]
        public string AdmissionNumber { get; set; } = string.Empty;

        public DateTime? AdmissionDate { get; set; }

        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(80)]
        public string FirstName { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(80)]
        public string LastName { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.StringLength(80)]
        public string? MiddleName { get; set; }

        public DateTime? DateOfBirth { get; set; }
        public Guid? GenderId { get; set; }
        public Guid? BloodGroupId { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(80)]
        public string? StateOfOrigin { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(300)]
        public string? ResidentialAddress { get; set; }
```

```
        [System.ComponentModel.DataAnnotations.StringLength(500)]
        public string? Allergies { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(1000)]
        public string? MedicalNotes { get; set; }
    }

    private class LinkFormModel
    {
        public Guid? ParentId { get; set; }
        public Guid? RelationshipId { get; set; }
        public bool IsPrimaryContact { get; set; }
        public bool CanPickUp { get; set; } = true;
        public string? Notes { get; set; }
    }

    private class EnrolmentFormModel
    {
        public Guid? ClassId { get; set; }
        public DateTime? EnrolledOn { get; set; } = DateTime.Today;
        public string? Notes { get; set; }
    }

    protected override async Task OnInitializedAsync()
    {
        genders = await LookupService.GetGendersAsync();
        bloodGroups = await LookupService.GetBloodGroupsAsync();
        relationships = await LookupService.GetRelationshipsAsync();
        parents = await LookupService.GetParentsAsync();
        allClasses = await LookupService.GetClassesForSessionAsync();

        await LoadStudentAsync();
        await LoadLinksAsync();
        await LoadEnrolmentsAsync();
    }

    private async Task LoadStudentAsync()
    {
        loadingProfile = true;
        try
        {
            student = await StudentService.GetByIdAsync(Id);
            if (student is not null)
            {
                form = new StudentFormModel
                {
                    AdmissionNumber = student.AdmissionNumber,
                    AdmissionDate = student.AdmissionDate.ToDateTime(TimeOnly.MinValue),
                    FirstName = student.FirstName,
                    LastName = student.LastName,
                    MiddleName = student.MiddleName,
                    DateOfBirth = student.DateOfBirth.ToDateTime(TimeOnly.MinValue),
                    GenderId = student.GenderId,
                    BloodGroupId = student.BloodGroupId,
```

```
                    StateOfOrigin = student.StateOfOrigin,
                    ResidentialAddress = student.ResidentialAddress,
                    Allergies = student.Allergies,
                    MedicalNotes = student.MedicalNotes,
                };
            }
        }
        finally
        {
            loadingProfile = false;
        }
    }

    private async Task LoadLinksAsync() =>
        parentLinks = await StudentService.GetParentLinksAsync(Id);

    private async Task LoadEnrolmentsAsync() =>
        enrolments = await EnrolmentService.ListAsync(new EnrolmentListFilter { StudentId =
Id });

    private async Task SaveAsync()
    {
        if (form.AdmissionDate is null || form.DateOfBirth is null) return;

        saving = true;
        try
        {
            var result = await StudentService.UpdateAsync(new UpdateStudentRequest
            {
                Id = Id,
                AdmissionNumber = form.AdmissionNumber,
                AdmissionDate = DateOnly.FromDateTime(form.AdmissionDate.Value),
                FirstName = form.FirstName,
                LastName = form.LastName,
                MiddleName = form.MiddleName,
                DateOfBirth = DateOnly.FromDateTime(form.DateOfBirth.Value),
                GenderId = form.GenderId,
                BloodGroupId = form.BloodGroupId,
                StateOfOrigin = form.StateOfOrigin,
                ResidentialAddress = form.ResidentialAddress,
                Allergies = form.Allergies,
                MedicalNotes = form.MedicalNotes,
            });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved", "Student profile
updated.");
                await LoadStudentAsync();
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
```

```
            }
        }
        finally
        {
            saving = false;
        }
    }

    private async Task SetActiveAsync(bool active)
    {
        var result = await StudentService.SetActiveAsync(Id, active,
CancellationToken.None);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Updated",
                active ? "Student activated." : "Student deactivated.");
            await LoadStudentAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }

    private void OpenLinkForm()
    {
        linkEditingId = null;
        linkForm = new LinkFormModel();
        showLinkForm = true;
    }

    private void OpenLinkEditForm(StudentParentDto link)
    {
        linkEditingId = link.Id;
        linkForm = new LinkFormModel
        {
            ParentId = link.ParentId,
            RelationshipId = link.RelationshipId,
            IsPrimaryContact = link.IsPrimaryContact,
            CanPickUp = link.CanPickUp,
            Notes = link.Notes,
        };
        showLinkForm = true;
    }

    private async Task SaveLinkAsync()
    {
        if (linkForm.RelationshipId is null
            || (!linkEditingId.HasValue && linkForm.ParentId is null))
        {
            Notification.Notify(NotificationSeverity.Warning, "Missing fields",
                "Pick a parent and a relationship.");
            return;
```

```
        }

        linkSaving = true;
        try
        {
            OperationResult result = linkEditingId.HasValue
                ? await StudentService.UpdateParentLinkAsync(new
UpdateStudentParentLinkRequest
                {
                    Id = linkEditingId.Value,
                    RelationshipId = linkForm.RelationshipId.Value,
                    IsPrimaryContact = linkForm.IsPrimaryContact,
                    CanPickUp = linkForm.CanPickUp,
                    Notes = linkForm.Notes,
                })
                : await StudentService.LinkParentAsync(new LinkStudentParentRequest
                {
                    StudentId = Id,
                    ParentId = linkForm.ParentId!.Value,
                    RelationshipId = linkForm.RelationshipId.Value,
                    IsPrimaryContact = linkForm.IsPrimaryContact,
                    CanPickUp = linkForm.CanPickUp,
                    Notes = linkForm.Notes,
                });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved", "Parent link
saved.");
                showLinkForm = false;
                await LoadLinksAsync();
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
            }
        }
        finally
        {
            linkSaving = false;
        }
    }

    private async Task UnlinkAsync(StudentParentDto link)
    {
        var confirm = await Dialog.Confirm(
            $"Unlink '{link.ParentName}' from this student?",
            "Unlink parent",
            new ConfirmOptions { OkButtonText = "Unlink", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await StudentService.UnlinkParentAsync(link.Id);
        if (result.Succeeded)
```

```
        {
            Notification.Notify(NotificationSeverity.Success, "Unlinked",
$"'{link.ParentName}' removed.");
            await LoadLinksAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }

    private void OpenEnrolmentForm()
    {
        enrolmentForm = new EnrolmentFormModel();
        showEnrolmentForm = true;
    }

    private async Task SaveEnrolmentAsync()
    {
        if (enrolmentForm.ClassId is null || enrolmentForm.EnrolledOn is null)
        {
            Notification.Notify(NotificationSeverity.Warning, "Missing fields",
                "Pick a class and enrolment date.");
            return;
        }

        enrolmentSaving = true;
        try
        {
            var result = await EnrolmentService.CreateAsync(new CreateEnrolmentRequest
            {
                StudentId = Id,
                SchoolClassId = enrolmentForm.ClassId.Value,
                EnrolledOn = DateOnly.FromDateTime(enrolmentForm.EnrolledOn.Value),
                Notes = enrolmentForm.Notes,
            });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Enrolled", "Pupil
enrolled in class.");
                showEnrolmentForm = false;
                await LoadEnrolmentsAsync();
                await LoadStudentAsync();
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
            }
        }
        finally
        {
```

```
            enrolmentSaving = false;
        }
    }

    private async Task WithdrawAsync(EnrolmentDto e)
    {
        var confirm = await Dialog.Confirm(
            $"Withdraw pupil from {e.SchoolClassName} ({e.SessionName})?",
            "Withdraw enrolment",
            new ConfirmOptions { OkButtonText = "Withdraw", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await EnrolmentService.WithdrawAsync(e.Id,
DateOnly.FromDateTime(DateTime.Today), null, CancellationToken.None);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Withdrawn", "Enrolment
withdrawn.");
            await LoadEnrolmentsAsync();
            await LoadStudentAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }

    private async Task DeleteEnrolmentAsync(EnrolmentDto e)
    {
        var confirm = await Dialog.Confirm(
            $"Delete enrolment in {e.SchoolClassName}? This cannot be reversed from the
UI.",
            "Delete enrolment",
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await EnrolmentService.SoftDeleteAsync(e.Id);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Deleted", "Enrolment
removed.");
            await LoadEnrolmentsAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }
}
```

Three patterns deserve a moment:

- Tab counts in titles. The tab labels read 'Parents (3)' and 'Enrolment history (5)' so an admin can see at a glance whether anything is there before they click. The numbers are reactive; they update when the underlying lists change.
- Inline link form. Clicking 'Link a parent' reveals a RadzenCard inside the Parents tab. The parent dropdown uses Radzen's AllowFiltering so admins can type the parent's name and pick from the search results — backed by LookupService.GetParentsAsync, capped at 50 results.
- Confirm-then-act for destructive operations. Every withdraw and delete goes through DialogService.Confirm with explicit OK/Cancel labels so a misclick does not lose data.

9.5 Parents.razor and CreateParent.razor

Parents.razor and CreateParent.razor mirror the sprint 2 and earlier-in-sprint-3 patterns exactly: a paged grid with search and status filters; a long inline form with title, names, marital status, contact details, and occupation. The grid surfaces the count of linked pupils per parent so the admin can spot 'directory-only' parents who never got linked to a child.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Family/Parents.razor

```
@page "/parents"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IParentService ParentService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Parents · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Parents &amp; guardians</h1>
        <p>The parents directory. Add new parents, edit contact details, then link them to
pupils from the student profile.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Add new parent" Icon="person_add"
ButtonStyle="ButtonStyle.Primary"
                      Click="@(() => Nav.NavigateTo("/parents/new"))" />
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Search</label>
            <RadzenTextBox @bind-Value="searchTerm" Placeholder="Name, phone, or email"
                           Change="@(_ => LoadAsync())" Style="min-width: 260px;" />
        </div>
        <div class="nps-field">
            <label>Status</label>
            <RadzenDropDown TValue="bool?" @bind-Value="isActive"
                            Data="@statusOptions" TextProperty="Text" ValueProperty="Value"
```

```
                            AllowClear="true" Placeholder="All"
                            Change="@(_ => LoadAsync())" Style="min-width: 160px;" />
        </div>
        <RadzenButton Text="Refresh" Icon="refresh" Variant="Variant.Flat"
                      ButtonStyle="ButtonStyle.Light" Click="@LoadAsync" />
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid @ref="grid" TItem="ParentDto" Data="@parents" Count="@total"
                    IsLoading="@loading" AllowPaging="true" AllowSorting="true"
                    PageSize="20" PageSizeOptions="@(new[] {10, 20, 50})"
                    LoadData="@OnLoadData" EmptyText="No parents match your filters.">
        <Columns>
            <RadzenDataGridColumn TItem="ParentDto" Title="Name" Property="FullName">
                <Template Context="p">
                    <strong>@(string.IsNullOrEmpty(p.TitleName) ? "" : p.TitleName + "
")@p.FullName</strong>
                    <br />
                    <small style="color: var(--nps-ink-500);">@p.Occupation</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ParentDto" Title="Contact" Sortable="false">
                <Template Context="p">
                    <div style="display:flex; flex-direction:column;">
                        <span>@p.PrimaryPhone</span>
                        <small style="color: var(--nps-ink-500);">@p.Email</small>
                    </div>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ParentDto" Title="Pupils" Property="StudentCount"
Width="100px" />
            <RadzenDataGridColumn TItem="ParentDto" Title="Status" Width="120px"
Property="IsActive">
                <Template Context="p">
                    @if (p.IsActive)
                    {
                        <RadzenBadge Text="Active" BadgeStyle="BadgeStyle.Success"
Variant="Variant.Flat" />
                    }
                    else
                    {
                        <RadzenBadge Text="Inactive" BadgeStyle="BadgeStyle.Secondary"
Variant="Variant.Flat" />
                    }
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="ParentDto" Title="Actions" Sortable="false"
Filterable="false"
                                  TextAlign="TextAlign.Right" Width="180px">
                <Template Context="p">
                    <div class="nps-inline-actions">
                        <RadzenButton Icon="edit" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Light" title="Edit"
```

```
                                        Click="@(() =>
Nav.NavigateTo($"/parents/{p.Id}"))" />
                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                        ButtonStyle="ButtonStyle.Danger" title="Delete"
                                        Click="@(() => DeleteAsync(p))" />
                    </div>
                </Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
</RadzenCard>

@code {
    private RadzenDataGrid<ParentDto>? grid;
    private List<ParentDto> parents = [];
    private int total;
    private bool loading;

    private string? searchTerm;
    private bool? isActive;

    private record StatusOption(string Text, bool? Value);
    private readonly List<StatusOption> statusOptions =
    [
        new StatusOption("Active", true),
        new StatusOption("Inactive", false),
    ];

    private int skip;
    private int take = 20;
    private string? orderBy;

    private async Task OnLoadData(LoadDataArgs args)
    {
        skip = args.Skip ?? 0;
        take = args.Top ?? 20;
        orderBy = args.OrderBy;
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            var result = await ParentService.ListAsync(new ParentListFilter
            {
                SearchTerm = searchTerm,
                IsActive = isActive,
                Skip = skip,
                Take = take,
                OrderBy = orderBy,
            });
```

```
            parents = result.Items;
            total = result.TotalCount;
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }

    private async Task DeleteAsync(ParentDto p)
    {
        var confirm = await Dialog.Confirm(
            $"Delete parent '{p.FullName}'? Linked pupils must be unlinked first.",
            "Delete parent",
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await ParentService.SoftDeleteAsync(p.Id);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Deleted", $"'{p.FullName}'
removed.");
            await LoadAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }
}
```

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Family/CreateParent.razor

```
@page "/parents/new"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IParentService ParentService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject NotificationService Notification

<PageTitle>New parent · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>New parent</h1>
        <p>Capture the parent's details. You can link pupils to them from any student
profile afterwards.</p>
    </div>
    <div class="nps-page-actions">
        <RadzenButton Text="Back to parents" Icon="arrow_back" Variant="Variant.Flat"
```

```
                        ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/parents"))" />
    </div>
</div>

<RadzenCard class="nps-card">
    <RadzenTemplateForm TItem="ParentFormModel" Data="@form" Submit="@SaveAsync">
        <DataAnnotationsValidator />
        <div class="nps-form-grid">
            <div class="nps-field">
                <label>Title</label>
                <RadzenDropDown TValue="Guid?" Data="@titles" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="form.TitleId" AllowClear="true"
Placeholder="Mr., Mrs., ..." />
            </div>
            <div class="nps-field">
                <label>Gender</label>
                <RadzenDropDown TValue="Guid?" Data="@genders" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="form.GenderId" AllowClear="true"
Placeholder="Pick one" />
            </div>
            <div class="nps-field">
                <label>First name *</label>
                <RadzenTextBox @bind-Value="form.FirstName" />
                <ValidationMessage For="() => form.FirstName" />
            </div>
            <div class="nps-field">
                <label>Last name *</label>
                <RadzenTextBox @bind-Value="form.LastName" />
                <ValidationMessage For="() => form.LastName" />
            </div>
            <div class="nps-field">
                <label>Middle name</label>
                <RadzenTextBox @bind-Value="form.MiddleName" />
            </div>
            <div class="nps-field">
                <label>Marital status</label>
                <RadzenDropDown TValue="Guid?" Data="@maritalStatuses" TextProperty="Name"
ValueProperty="Id"
                                @bind-Value="form.MaritalStatusId" AllowClear="true"
Placeholder="Pick one" />
            </div>
            <div class="nps-field">
                <label>Primary phone</label>
                <RadzenTextBox @bind-Value="form.PrimaryPhone" Placeholder="+234..." />
            </div>
            <div class="nps-field">
                <label>Alternate phone</label>
                <RadzenTextBox @bind-Value="form.AlternatePhone" />
            </div>
            <div class="nps-field nps-span-2">
                <label>Email</label>
                <RadzenTextBox @bind-Value="form.Email" Placeholder="parent@example.com" />
```

```
            </div>
            <div class="nps-field nps-span-2">
                <label>Residential address</label>
                <RadzenTextArea @bind-Value="form.ResidentialAddress" Rows="2" />
            </div>
            <div class="nps-field">
                <label>Occupation</label>
                <RadzenTextBox @bind-Value="form.Occupation" />
            </div>
            <div class="nps-field">
                <label>Employer</label>
                <RadzenTextBox @bind-Value="form.Employer" />
            </div>
        </div>
        <div class="nps-form-actions">
            <RadzenButton Text="Cancel" ButtonType="ButtonType.Button"
                          ButtonStyle="ButtonStyle.Light" Variant="Variant.Flat"
                          Click="@(() => Nav.NavigateTo("/parents"))" />
            <RadzenButton Text="@(saving ? "Saving..." : "Save parent")"
                          Icon="save" ButtonType="ButtonType.Submit"
                          ButtonStyle="ButtonStyle.Primary" Disabled="@saving" />
        </div>
    </RadzenTemplateForm>
</RadzenCard>

@code {
    private ParentFormModel form = new();
    private bool saving;

    private IReadOnlyList<LookupDto> titles = [];
    private IReadOnlyList<LookupDto> genders = [];
    private IReadOnlyList<LookupDto> maritalStatuses = [];

    private class ParentFormModel
    {
        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(80)]
        public string FirstName { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(80)]
        public string LastName { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.StringLength(80)]
        public string? MiddleName { get; set; }

        public Guid? TitleId { get; set; }
        public Guid? GenderId { get; set; }
        public Guid? MaritalStatusId { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(30)]
        public string? PrimaryPhone { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(30)]
```

```
        public string? AlternatePhone { get; set; }

        [System.ComponentModel.DataAnnotations.EmailAddress,
System.ComponentModel.DataAnnotations.StringLength(256)]
        public string? Email { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(300)]
        public string? ResidentialAddress { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(120)]
        public string? Occupation { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(120)]
        public string? Employer { get; set; }
    }

    protected override async Task OnInitializedAsync()
    {
        titles = await LookupService.GetTitlesAsync();
        genders = await LookupService.GetGendersAsync();
        maritalStatuses = await LookupService.GetMaritalStatusesAsync();
    }

    private async Task SaveAsync()
    {
        saving = true;
        try
        {
            var result = await ParentService.CreateAsync(new CreateParentRequest
            {
                FirstName = form.FirstName,
                LastName = form.LastName,
                MiddleName = form.MiddleName,
                TitleId = form.TitleId,
                GenderId = form.GenderId,
                MaritalStatusId = form.MaritalStatusId,
                PrimaryPhone = form.PrimaryPhone,
                AlternatePhone = form.AlternatePhone,
                Email = form.Email,
                ResidentialAddress = form.ResidentialAddress,
                Occupation = form.Occupation,
                Employer = form.Employer,
            });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved",
                    $"Parent '{form.FirstName} {form.LastName}' created.");
                Nav.NavigateTo($"/parents/{result.Data}");
            }
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
```

```
            }
        }
        finally
        {
            saving = false;
        }
    }
}
```

9.6 EditParent.razor — the parent detail view

EditParent.razor uses the same tabs pattern as EditStudent. The Profile tab is the editable form; the Linked pupils tab is read-only — it shows every pupil this parent is linked to, with a click-through to the pupil. Linking is initiated from the pupil's Edit page rather than from here, because in practice an admin is almost always thinking 'this pupil needs another parent' rather than 'this parent needs more pupils'.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Family/EditParent.razor

```
@page "/parents/{Id:guid}"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IParentService ParentService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject NotificationService Notification

<PageTitle>Edit parent · Naija Prime School</PageTitle>

@if (loading)
{
    <p>Loading parent...</p>
}
else if (parent is null)
{
    <RadzenAlert Severity="AlertSeverity.Warning">Parent not found.</RadzenAlert>
}
else
{
    <div class="nps-page-header">
        <div>
            <h1>@(string.IsNullOrEmpty(parent.TitleName) ? "" : parent.TitleName + "
")@parent.FullName</h1>
            <p>@parent.PrimaryPhone @(string.IsNullOrEmpty(parent.Email) ? "" : "· " +
parent.Email)</p>
        </div>
        <div class="nps-page-actions">
            @if (parent.IsActive)
            {
                <RadzenButton Text="Deactivate" Icon="block" Variant="Variant.Flat"
                              ButtonStyle="ButtonStyle.Warning" Click="@(() =>
SetActiveAsync(false))" />
            }
            else
```

```
            {
                <RadzenButton Text="Activate" Icon="check_circle" Variant="Variant.Flat"
                              ButtonStyle="ButtonStyle.Success" Click="@(() =>
SetActiveAsync(true))" />
            }
            <RadzenButton Text="Back" Icon="arrow_back" Variant="Variant.Flat"
                          ButtonStyle="ButtonStyle.Light" Click="@(() =>
Nav.NavigateTo("/parents"))" />
        </div>
    </div>

    <RadzenTabs @bind-SelectedIndex="selectedTab">
        <Tabs>
            <RadzenTabsItem Text="Profile">
                <RadzenCard class="nps-card">
                    <RadzenTemplateForm TItem="ParentFormModel" Data="@form"
Submit="@SaveAsync">
                        <DataAnnotationsValidator />
                        <div class="nps-form-grid">
                            <div class="nps-field">
                                <label>Title</label>
                                <RadzenDropDown TValue="Guid?" Data="@titles"
TextProperty="Name" ValueProperty="Id"
                                                @bind-Value="form.TitleId"
AllowClear="true" />
                            </div>
                            <div class="nps-field">
                                <label>Gender</label>
                                <RadzenDropDown TValue="Guid?" Data="@genders"
TextProperty="Name" ValueProperty="Id"
                                                @bind-Value="form.GenderId"
AllowClear="true" />
                            </div>
                            <div class="nps-field">
                                <label>First name *</label>
                                <RadzenTextBox @bind-Value="form.FirstName" />
                            </div>
                            <div class="nps-field">
                                <label>Last name *</label>
                                <RadzenTextBox @bind-Value="form.LastName" />
                            </div>
                            <div class="nps-field">
                                <label>Middle name</label>
                                <RadzenTextBox @bind-Value="form.MiddleName" />
                            </div>
                            <div class="nps-field">
                                <label>Marital status</label>
                                <RadzenDropDown TValue="Guid?" Data="@maritalStatuses"
TextProperty="Name" ValueProperty="Id"
                                                @bind-Value="form.MaritalStatusId"
AllowClear="true" />
                            </div>
                            <div class="nps-field">
                                <label>Primary phone</label>
                                <RadzenTextBox @bind-Value="form.PrimaryPhone" />
```

```
                            </div>
                            <div class="nps-field">
                                <label>Alternate phone</label>
                                <RadzenTextBox @bind-Value="form.AlternatePhone" />
                            </div>
                            <div class="nps-field nps-span-2">
                                <label>Email</label>
                                <RadzenTextBox @bind-Value="form.Email" />
                            </div>
                            <div class="nps-field nps-span-2">
                                <label>Residential address</label>
                                <RadzenTextArea @bind-Value="form.ResidentialAddress"
Rows="2" />
                            </div>
                            <div class="nps-field">
                                <label>Occupation</label>
                                <RadzenTextBox @bind-Value="form.Occupation" />
                            </div>
                            <div class="nps-field">
                                <label>Employer</label>
                                <RadzenTextBox @bind-Value="form.Employer" />
                            </div>
                        </div>
                        <div class="nps-form-actions">
                            <RadzenButton Text="@(saving ? "Saving..." : "Save changes")"
                                          Icon="save" ButtonType="ButtonType.Submit"
                                          ButtonStyle="ButtonStyle.Primary"
Disabled="@saving" />
                        </div>
                    </RadzenTemplateForm>
                </RadzenCard>
            </RadzenTabsItem>

            <RadzenTabsItem Text="@($"Linked pupils ({studentLinks.Count})")">
                <RadzenCard class="nps-card">
                    <p style="color: var(--nps-ink-500);">
                        Use the student profile (Family → Students → pick a pupil) to add
or remove links.
                    </p>
                    <RadzenDataGrid TItem="StudentParentDto" Data="@studentLinks"
AllowPaging="false"
                                    EmptyText="No pupils linked to this parent yet.">
                        <Columns>
                            <RadzenDataGridColumn TItem="StudentParentDto" Title="Pupil">
                                <Template Context="l">
                                    <strong>@l.StudentName</strong>
                                    <br />
                                    <small style="color: var(--nps-
ink-500);">@l.StudentAdmissionNumber</small>
                                </Template>
                            </RadzenDataGridColumn>
                            <RadzenDataGridColumn TItem="StudentParentDto"
Title="Relationship" Property="RelationshipName" Width="160px" />
                            <RadzenDataGridColumn TItem="StudentParentDto" Title="Primary"
Width="100px">
```

```
                                <Template Context="l">
                                    @if (l.IsPrimaryContact)
                                    {
                                        <RadzenBadge Text="Yes"
BadgeStyle="BadgeStyle.Success" Variant="Variant.Flat" />
                                    }
                                </Template>
                            </RadzenDataGridColumn>
                            <RadzenDataGridColumn TItem="StudentParentDto" Title="Pickup"
Width="100px">
                                <Template Context="l">
                                    @if (l.CanPickUp)
                                    {
                                        <RadzenBadge Text="Yes"
BadgeStyle="BadgeStyle.Success" Variant="Variant.Flat" />
                                    }
                                    else
                                    {
                                        <RadzenBadge Text="No"
BadgeStyle="BadgeStyle.Warning" Variant="Variant.Flat" />
                                    }
                                </Template>
                            </RadzenDataGridColumn>
                            <RadzenDataGridColumn TItem="StudentParentDto" Title="Open"
                                                  Width="120px" Sortable="false"
TextAlign="TextAlign.Right">
                                <Template Context="l">
                                    <RadzenButton Icon="open_in_new"
Size="ButtonSize.Small" Variant="Variant.Flat"
                                                  ButtonStyle="ButtonStyle.Light"
title="Open student"
                                                  Click="@(() =>
Nav.NavigateTo($"/students/{l.StudentId}"))" />
                                </Template>
                            </RadzenDataGridColumn>
                        </Columns>
                    </RadzenDataGrid>
                </RadzenCard>
            </RadzenTabsItem>
        </Tabs>
    </RadzenTabs>
}

@code {
    [Parameter] public Guid Id { get; set; }

    private bool loading = true;
    private bool saving;
    private int selectedTab;

    private ParentDto? parent;
    private ParentFormModel form = new();

    private IReadOnlyList<LookupDto> titles = [];
```

```
    private IReadOnlyList<LookupDto> genders = [];
    private IReadOnlyList<LookupDto> maritalStatuses = [];
    private IReadOnlyList<StudentParentDto> studentLinks = [];

    private class ParentFormModel
    {
        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(80)]
        public string FirstName { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.Required,
System.ComponentModel.DataAnnotations.StringLength(80)]
        public string LastName { get; set; } = string.Empty;

        [System.ComponentModel.DataAnnotations.StringLength(80)]
        public string? MiddleName { get; set; }

        public Guid? TitleId { get; set; }
        public Guid? GenderId { get; set; }
        public Guid? MaritalStatusId { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(30)]
        public string? PrimaryPhone { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(30)]
        public string? AlternatePhone { get; set; }

        [System.ComponentModel.DataAnnotations.EmailAddress,
System.ComponentModel.DataAnnotations.StringLength(256)]
        public string? Email { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(300)]
        public string? ResidentialAddress { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(120)]
        public string? Occupation { get; set; }

        [System.ComponentModel.DataAnnotations.StringLength(120)]
        public string? Employer { get; set; }
    }

    protected override async Task OnInitializedAsync()
    {
        titles = await LookupService.GetTitlesAsync();
        genders = await LookupService.GetGendersAsync();
        maritalStatuses = await LookupService.GetMaritalStatusesAsync();
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
```

```
            parent = await ParentService.GetByIdAsync(Id);
            if (parent is not null)
            {
                form = new ParentFormModel
                {
                    FirstName = parent.FirstName,
                    LastName = parent.LastName,
                    MiddleName = parent.MiddleName,
                    TitleId = parent.TitleId,
                    GenderId = parent.GenderId,
                    MaritalStatusId = parent.MaritalStatusId,
                    PrimaryPhone = parent.PrimaryPhone,
                    AlternatePhone = parent.AlternatePhone,
                    Email = parent.Email,
                    ResidentialAddress = parent.ResidentialAddress,
                    Occupation = parent.Occupation,
                    Employer = parent.Employer,
                };
                studentLinks = await ParentService.GetStudentLinksAsync(Id);
            }
        }
        finally
        {
            loading = false;
        }
    }

    private async Task SaveAsync()
    {
        saving = true;
        try
        {
            var result = await ParentService.UpdateAsync(new UpdateParentRequest
            {
                Id = Id,
                FirstName = form.FirstName,
                LastName = form.LastName,
                MiddleName = form.MiddleName,
                TitleId = form.TitleId,
                GenderId = form.GenderId,
                MaritalStatusId = form.MaritalStatusId,
                PrimaryPhone = form.PrimaryPhone,
                AlternatePhone = form.AlternatePhone,
                Email = form.Email,
                ResidentialAddress = form.ResidentialAddress,
                Occupation = form.Occupation,
                Employer = form.Employer,
            });

            if (result.Succeeded)
            {
                Notification.Notify(NotificationSeverity.Success, "Saved", "Parent profile
updated.");
                await LoadAsync();
            }
```

```
            else
            {
                Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
            }
        }
        finally
        {
            saving = false;
        }
    }

    private async Task SetActiveAsync(bool active)
    {
        var result = await ParentService.SetActiveAsync(Id, active,
CancellationToken.None);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Updated",
                active ? "Parent activated." : "Parent deactivated.");
            await LoadAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }
}
```

9.7 Enrolments.razor — the overview

Enrolments.razor is the only sprint-3 page that does not drill into a single pupil. It is the registrar's view: pick a session and a class (or a status), see every pupil whose enrolment matches. Withdraw and delete actions live inline on each row; opening a pupil pops their EditStudent page in the same browser tab.

Listing — src/NaijaPrimeSchool.Web/Components/Pages/Family/Enrolments.razor

```
@page "/enrolments"
@attribute [Authorize(Roles = $"{Roles.SuperAdmin},{Roles.HeadTeacher}")]
@rendermode InteractiveServer

@inject IEnrolmentService EnrolmentService
@inject ISessionService SessionService
@inject ILookupService LookupService
@inject NavigationManager Nav
@inject DialogService Dialog
@inject NotificationService Notification

<PageTitle>Enrolments · Naija Prime School</PageTitle>

<div class="nps-page-header">
    <div>
        <h1>Enrolments</h1>
```

```
        <p>Browse pupils by session and class. Withdraw, switch, or delete enrolments
inline.</p>
    </div>
</div>

<RadzenCard class="nps-card" Style="margin-bottom: 1rem;">
    <div class="nps-filter-bar">
        <div class="nps-field">
            <label>Session</label>
            <RadzenDropDown TValue="Guid?" Data="@sessions" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="sessionId" AllowClear="true" Placeholder="All
sessions"
                            Change="@(_ => SessionChangedAsync())" Style="min-width:
200px;" />
        </div>
        <div class="nps-field">
            <label>Class</label>
            <RadzenDropDown TValue="Guid?" Data="@classes" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="classId" AllowClear="true" Placeholder="All
classes"
                            Change="@(_ => LoadAsync())" Style="min-width: 240px;" />
        </div>
        <div class="nps-field">
            <label>Status</label>
            <RadzenDropDown TValue="Guid?" Data="@statuses" TextProperty="Name"
ValueProperty="Id"
                            @bind-Value="statusId" AllowClear="true" Placeholder="All
statuses"
                            Change="@(_ => LoadAsync())" Style="min-width: 200px;" />
        </div>
        <RadzenButton Text="Refresh" Icon="refresh" Variant="Variant.Flat"
                      ButtonStyle="ButtonStyle.Light" Click="@LoadAsync" />
    </div>
</RadzenCard>

<RadzenCard class="nps-card">
    <RadzenDataGrid TItem="EnrolmentDto" Data="@enrolments" IsLoading="@loading"
                    AllowPaging="true" PageSize="25" EmptyText="No enrolments match your
filters.">
        <Columns>
            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Pupil">
                <Template Context="e">
                    <strong>@e.StudentName</strong>
                    <br />
                    <small style="color:
var(--nps-ink-500);">@e.StudentAdmissionNumber</small>
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Class">
                <Template Context="e">
                    <strong>@e.SchoolClassName</strong>
                    <br />
                    <small style="color: var(--nps-ink-500);">@e.SessionName</small>
```

```
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Enrolled"
Property="EnrolledOn"
                                  FormatString="{0:d MMM yyyy}" Width="140px" />
            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Withdrawn" Width="140px">
                <Template Context="e">
                    @(e.WithdrawnOn.HasValue ? e.WithdrawnOn.Value.ToString("d MMM yyyy") :
"—")
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Status" Width="140px">
                <Template Context="e">
                    <RadzenBadge Text="@e.EnrolmentStatusName"
                                 BadgeStyle="@(e.WithdrawnOn.HasValue ?
BadgeStyle.Secondary : BadgeStyle.Success)"
                                 Variant="Variant.Flat" />
                </Template>
            </RadzenDataGridColumn>
            <RadzenDataGridColumn TItem="EnrolmentDto" Title="Actions" Sortable="false"
                                  Width="200px" TextAlign="TextAlign.Right">
                <Template Context="e">
                    <div class="nps-inline-actions">
                        <RadzenButton Icon="open_in_new" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Light" title="Open student"
                                      Click="@(() =>
Nav.NavigateTo($"/students/{e.StudentId}"))" />
                        @if (!e.WithdrawnOn.HasValue)
                        {
                            <RadzenButton Icon="logout" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                          ButtonStyle="ButtonStyle.Warning"
title="Withdraw"
                                          Click="@(() => WithdrawAsync(e))" />
                        }
                        <RadzenButton Icon="delete" Size="ButtonSize.Small"
Variant="Variant.Flat"
                                      ButtonStyle="ButtonStyle.Danger" title="Delete"
                                      Click="@(() => DeleteAsync(e))" />
                    </div>
                </Template>
            </RadzenDataGridColumn>
        </Columns>
    </RadzenDataGrid>
</RadzenCard>

@code {
    private IReadOnlyList<EnrolmentDto> enrolments = [];
    private IReadOnlyList<SessionDto> sessions = [];
    private IReadOnlyList<LookupDto> classes = [];
    private IReadOnlyList<LookupDto> statuses = [];
    private bool loading;

    private Guid? sessionId;
```

```
    private Guid? classId;
    private Guid? statusId;

    protected override async Task OnInitializedAsync()
    {
        sessions = await SessionService.ListAsync();
        sessionId = sessions.FirstOrDefault(s => s.IsCurrent)?.Id;
        classes = await LookupService.GetClassesForSessionAsync(sessionId);
        statuses = await LookupService.GetEnrolmentStatusesAsync();
        await LoadAsync();
    }

    private async Task SessionChangedAsync()
    {
        classId = null;
        classes = await LookupService.GetClassesForSessionAsync(sessionId);
        await LoadAsync();
    }

    private async Task LoadAsync()
    {
        loading = true;
        try
        {
            enrolments = await EnrolmentService.ListAsync(new EnrolmentListFilter
            {
                SessionId = sessionId,
                SchoolClassId = classId,
                EnrolmentStatusId = statusId,
            });
        }
        finally
        {
            loading = false;
            StateHasChanged();
        }
    }

    private async Task WithdrawAsync(EnrolmentDto e)
    {
        var confirm = await Dialog.Confirm(
            $"Withdraw {e.StudentName} from {e.SchoolClassName}?",
            "Withdraw enrolment",
            new ConfirmOptions { OkButtonText = "Withdraw", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await EnrolmentService.WithdrawAsync(e.Id,
DateOnly.FromDateTime(DateTime.Today), null, CancellationToken.None);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Withdrawn", "Enrolment
withdrawn.");
            await LoadAsync();
        }
```

```
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }

    private async Task DeleteAsync(EnrolmentDto e)
    {
        var confirm = await Dialog.Confirm(
            $"Delete enrolment of {e.StudentName} in {e.SchoolClassName}?",
            "Delete enrolment",
            new ConfirmOptions { OkButtonText = "Delete", CancelButtonText = "Cancel" });
        if (confirm != true) return;

        var result = await EnrolmentService.SoftDeleteAsync(e.Id);
        if (result.Succeeded)
        {
            Notification.Notify(NotificationSeverity.Success, "Deleted", "Enrolment
removed.");
            await LoadAsync();
        }
        else
        {
            Notification.Notify(NotificationSeverity.Error, "Failed", string.Join("; ",
result.Errors));
        }
    }
}
```

Note that this page does not have its own create form — creating an enrolment is done from EditStudent's Enrolment history tab, which has access to the pupil context. Enrolling a pupil from the overview page would mean adding a Student dropdown to the form and that quickly becomes fiddly with thousands of pupils. Schools that need a bulk-enrolment workflow will get a dedicated screen in a later sprint.

10. Navigation, imports, and authorization

10.1 NavMenu — a new Family panel

NavMenu.razor picks up a fourth role-gated panel between Academics and the Finance/Inventory placeholders. Five entries live inside it: Students, Add Student, Parents, Add Parent, Enrolments. The whole panel is wrapped in an AuthorizeView that requires SuperAdmin or HeadTeacher.

Excerpt — NavMenu.razor

```
            <RadzenPanelMenuItem Text="Family" Icon="family_restroom" Expanded="true">
                <RadzenPanelMenuItem Text="Students" Icon="boy" Path="/students" />
                <RadzenPanelMenuItem Text="Add Student" Icon="person_add"
Path="/students/new" />
                <RadzenPanelMenuItem Text="Parents" Icon="people_outline"
Path="/parents" />
                <RadzenPanelMenuItem Text="Add Parent" Icon="group_add" Path="/parents/new"
/>
                <RadzenPanelMenuItem Text="Enrolments" Icon="how_to_reg" Path="/enrolments"
/>
            </RadzenPanelMenuItem>
        </Authorized>
    </AuthorizeView>
```

10.2 _Imports.razor

_Imports.razor picks up two new @using lines so the family DTOs and service interfaces are visible to every Razor file in the Web project without per-file imports.

Excerpt — _Imports.razor

```
@using NaijaPrimeSchool.Application.Family
@using NaijaPrimeSchool.Application.Family.Dtos
```

10.3 Authorization at the page level

Every sprint-3 page declares an [Authorize] attribute restricting access to SuperAdmin + HeadTeacher. There is no policy added — the existing ManageUsers policy is intended for SuperAdmin only, and HeadTeacher access here is a permission specific to the family pages, not a general role promotion. Future sprints (attendance, results) will likely introduce a 'Pastoral' policy that includes class teachers; this sprint deliberately stays narrow.

11. Lifecycle of a family-domain row

Walking a single pupil from creation to withdrawal-and-deletion is the clearest way to see how all the layers cooperate. Imagine an admin admitting a six-year-old named Adaeze Okonkwo into Primary 1A for the 2025/2026 session.

11.1 Admin opens /students/new

- Authorization fires: the user must be SuperAdmin or HeadTeacher. If they are not, ASP.NET Core returns 403.
- OnInitializedAsync calls LookupService.GetGendersAsync, GetBloodGroupsAsync, and GetClassesForSessionAsync(currentSession.Id). Three database hits, all on tiny tables.
- The page renders the long inline form with three sections.

11.2 Admin fills the form and clicks Save

- RadzenTemplateForm runs DataAnnotations validation. If anything is missing, ValidationMessage components render inline.
- If the form is valid, SaveAsync converts the DateTime?-bound dates to DateOnly and calls StudentService.CreateAsync(request).
- StudentService.CreateAsync re-runs the cross-aggregate checks (DOB before admission date, AdmissionNumber unique, InitialClassId exists if provided), constructs a Student, and — if InitialClassId was provided — also constructs an Enrolment with EnrolledOn = AdmissionDate and the lowest-DisplayOrder EnrolmentStatus.
- db.Students.Add and db.Enrolments.Add stage both rows; SaveChangesAsync commits them in one transaction.
- ApplicationDbContext.SaveChanges (sprint 1) stamps CreatedOn/By on both rows.
- On success, the page navigates to /students/{newId} so the admin lands on Adaeze's profile and can immediately go to the Parents tab.

11.3 Admin links Adaeze's mother

- Parents tab loads via StudentService.GetParentLinksAsync — empty, since Adaeze just got created.
- Admin clicks Link a parent. The inline form appears.
- Admin types 'Okonkwo' into the parent dropdown; AllowFiltering kicks in and LookupService.GetParentsAsync("Okonkwo") returns the candidates, capped at 50.
- Admin picks Mrs Ngozi Okonkwo, picks Mother as the relationship, leaves Primary contact ON and Authorised to collect ON, and clicks Save link.
- StudentService.LinkParentAsync flips IsPrimaryContact = false on every other link for this student (none, in this case) and inserts the new StudentParent row.
- On success the form closes and GetParentLinksAsync re-runs, so the grid shows the new link with the 'Primary' badge.

11.4 Three years later, Adaeze withdraws

- Admin opens Adaeze's profile and goes to Enrolment history. Three rows are visible: Primary 1A 2025/2026 (withdrawn at end of session), Primary 2A 2026/2027 (withdrawn similarly), Primary 3A 2027/2028 (still active).

- Admin clicks the Withdraw icon on the active row.
- DialogService.Confirm asks for confirmation. Admin clicks Withdraw.
- EnrolmentService.WithdrawAsync looks up the 'Withdrawn' status row, sets WithdrawnOn = today, EnrolmentStatusId = Withdrawn, appends notes.
- ApplicationDbContext.SaveChanges stamps ModifiedOn/By on the row.
- The grid re-renders; the row's status badge flips to 'Withdrawn' and the Withdraw icon disappears (the action is hidden once WithdrawnOn is set).

11.5 The pupil leaves the school

- Admin clicks Delete on the pupil. DialogService.Confirm asks for confirmation.
- StudentService.SoftDeleteAsync loads the student with .Include(s => s.Enrolments).Include(s => s.ParentLinks). It checks for any enrolment with WithdrawnOn IS NULL — there is none, so the delete is allowed.
- db.Students.Remove(student); SaveChangesAsync(). The SaveChanges override intercepts the Deleted state, flips IsDeleted = true and stamps DeletedOn/By. The row is now invisible to ordinary queries because of the global query filter, but every audit column is preserved.

12. Smoke-test walkthrough

Once the build is green and the migration has applied, here is the end-to-end smoke test you can run against a fresh checkout. It covers the happy paths and a couple of important error paths.

12.1 Build, migrate, run

```
dotnet restore
dotnet build NaijaPrimeSchool.slnx
dotnet run --project src/NaijaPrimeSchool.Web
```

The first run applies migrations and seeds the four new lookup tables. Sign in as superadmin@naijaprimeschool.ng / Admin@12345.

12.2 Verify navigation

- The Family panel appears in the sidebar under Academics.
- Family contains five items: Students, Add Student, Parents, Add Parent, Enrolments.
- Click each one — every page renders without error.

12.3 Create a parent

11. Family → Add Parent.
12. Pick title Mrs., gender Female, marital status Married.
13. Fill First name 'Ngozi', Last name 'Okonkwo'.
14. Primary phone 080-xxx-xxxx, email ngozi.okonkwo@example.com.
15. Save. You should land on the parent edit page.

12.4 Create a pupil and enrol them

16. Family → Add Student.
17. Admission number NPS/2026/0001, admission date today, first name Adaeze, last name Okonkwo, DOB any date six years ago.
18. Pick gender Female, blood group O+, leave allergies blank.
19. In Initial enrolment, pick a class from the current session (if you do not have one, hop to /classes and create Primary 1A first).
20. Save. You land on Adaeze's profile.

12.5 Link the parent

21. Click the Parents tab.
22. Click Link a parent.
23. Pick Ngozi Okonkwo from the dropdown, relationship Mother, primary contact ON, can pick up ON.
24. Save link. The grid shows Ngozi with the 'Primary' badge.

12.6 Verify the directory views

- Family → Students. Adaeze is in the grid; the Primary contact column shows Ngozi Okonkwo and her phone number.
- Family → Parents. Ngozi is in the grid; the Pupils column shows 1.

- Family → Enrolments. Filter to current session; Adaeze's enrolment in Primary 1A is visible with status Active.

12.7 Verify the error paths

25. Try to delete Ngozi from /parents. The OperationResult comes back with 'Cannot delete a parent who is still linked to a student. Unlink them first.'
26. Try to delete Adaeze from /students without first withdrawing her enrolment. The OperationResult comes back with 'Cannot delete a student with an active enrolment.'
27. Withdraw Adaeze from her enrolment. Now you can delete the pupil if you choose. Don't actually do it for the demo.

12.8 Verify soft-delete and audit

Connect to SQL Server and run:

```
SELECT Id, FirstName, LastName, IsDeleted, DeletedOn, DeletedBy
FROM Students;

SELECT Id, EnrolledOn, WithdrawnOn, IsDeleted
FROM Enrolments;
```

If you withdrew and then deleted, IsDeleted is true on the Students row, DeletedOn and DeletedBy are populated, but the row is still there. The Enrolments row is also soft-deleted (because EnrolmentService.SoftDeleteAsync was called) — but only if you triggered the Delete; withdrawing alone does not delete.

13. Troubleshooting and gotchas

13.1 'A student with admission number X already exists'

Admission numbers are unique. If you copy-paste from an existing pupil and forget to bump the suffix, the service rejects it with a friendly message before the database throws. The unique index on Students.AdmissionNumber is the ultimate authority.

13.2 'Student already has an active enrolment in this session'

A pupil cannot be in two classes in the same session. If they need to move from Primary 1A to Primary 1B mid-term, withdraw the first enrolment, then create the second. The history view will show both rows.

13.3 'A given parent is already linked to the student'

The (StudentId, ParentId) unique index prevents duplicate links. If you want to change the relationship of an existing link, edit it (the pencil icon) rather than creating a new one.

13.4 The parent dropdown is empty

GetParentsAsync caps at 50 results without a search term. If your school has more than 50 parents and the one you want is alphabetically late, type a few letters of their name into the dropdown — Radzen's AllowFiltering re-queries with the term and the right parent will appear.

13.5 The classes dropdown is empty when creating a student

GetClassesForSessionAsync defaults to the current session. If no session is marked current, the call returns every class in the database, which is fine. If neither is true — no current session and no classes anywhere — you need to create a session and at least one class first via /sessions and /classes (sprint 2 pages).

13.6 The 'Cannot delete' message persists after I cleared dependents

EF Core's change tracker is per-DbContext-instance, and Blazor Server pages share their context across the page lifecycle. If you deleted the dependent rows in another tab, refresh the Students or Parents page so the new context reads the current state.

13.7 The Identity / soft-delete migration warning

The EF Core warning about ApplicationRole's query filter and IdentityUserRole's required end is harmless at runtime and predates sprint 3. We chose not to silence it because a future role-management UI will want to revisit it. The build is still green.

14. Forward-compatibility, today

Sprint 3 has deliberately left a few breadcrumbs that make the next sprints' work cleaner.

- Student.UserId and Parent.UserId are nullable hooks for future portal logins. The filtered unique indexes make wiring up an Identity user safe — at most one ApplicationUser per pupil/parent — without forcing every existing row to have one.
- Enrolment.WithdrawnOn distinguishes 'historical record' from 'current state'. Attendance (sprint 4) will read 'open enrolments' as 'WithdrawnOn IS NULL', which is the load-bearing definition.
- StudentParent.IsPrimaryContact is the column the parent portal sprint will use to decide who gets the 'home' homepage when both parents have logins.
- BloodGroup, Allergies and MedicalNotes are captured but not yet surfaced anywhere except the pupil profile. The school nurse sprint (or the attendance sprint, depending on scheduling) will read those fields.
- Enrolment.SchoolClassId joining through to Session means every fee/result/attendance feature can navigate from a pupil to their session in one join — no Enrolment.SessionId duplication.

14.1 What might need a small refactor later

- GetStudentsAsync and GetParentsAsync cap at 50 rows. Once the school exceeds 5,000 pupils, server-side virtual scrolling on the dropdowns will be worth the engineering.
- StudentService is taking on parent-linkage methods. If a future feature (sibling discovery, family-wide messaging) warrants a dedicated IStudentParentService, those four methods can be extracted in a single PR.
- The Student / Parent denormalised name columns will need to stay in sync if the portal sprint introduces a 'change your name' flow on the pupil's account. The simplest approach is to update both rows in the same SaveChanges; we will add a tiny helper when that flow lands.
- Bulk import (CSV/Excel of pupils) is a feature-add that the schema already supports. The lift is a service method that loops over rows and calls CreateAsync, plus a Razor page with file upload.

15. Appendix — files added or changed in sprint 3

Domain layer (new)	—
src/NaijaPrimeSchool.Domain/Family/Relationship.cs	Lookup.
src/NaijaPrimeSchool.Domain/Family/EnrolmentStatus.cs	Lookup.
src/NaijaPrimeSchool.Domain/Family/BloodGroup.cs	Lookup.
src/NaijaPrimeSchool.Domain/Family/MaritalStatus.cs	Lookup.
src/NaijaPrimeSchool.Domain/Family/Student.cs	Pupil entity.
src/NaijaPrimeSchool.Domain/Family/Parent.cs	Parent / guardian entity.
src/NaijaPrimeSchool.Domain/Family/StudentParent.cs	Linkage with relationship + flags.
src/NaijaPrimeSchool.Domain/Family/Enrolment.cs	(student × class) over time.
Domain layer (modified)	—
src/NaijaPrimeSchool.Domain/Academics/SchoolClass.cs	Added ICollection<Enrolment> Enrolments.
Application layer (new)	—
src/NaijaPrimeSchool.Application/Family/Dtos/StudentDtos.cs	Student DTOs.
src/NaijaPrimeSchool.Application/Family/Dtos/ParentDtos.cs	Parent DTOs.
src/NaijaPrimeSchool.Application/Family/Dtos/StudentParentDtos.cs	Linkage DTOs.
src/NaijaPrimeSchool.Application/Family/Dtos/EnrolmentDtos.cs	Enrolment DTOs.
src/NaijaPrimeSchool.Application/Family/IStudentService.cs	Student service contract (CRUD + linkage).
src/NaijaPrimeSchool.Application/Family/IParentService.cs	Parent service contract.
src/NaijaPrimeSchool.Application/Family/IEnrolmentService.cs	Enrolment service contract.
Application layer (modified)	—
src/NaijaPrimeSchool.Application/Users/ILookupService.cs	Added 7 new lookup methods.
Infrastructure layer (new)	—
src/NaijaPrimeSchool.Infrastructure/Services/StudentService.cs	Student CRUD + linkage methods.
src/NaijaPrimeSchool.Infrastructure/Services/ParentService.cs	Parent CRUD.
src/NaijaPrimeSchool.Infrastructure/Services/EnrolmentService.cs	Enrolment CRUD + Withdraw.
src/NaijaPrimeSchool.Infrastructure/Persistence/Migrations/20260429211125_StudentsAndParents.cs	EF migration adding 8 tables and 16 indexes.

Infrastructure layer (modified)	—
src/NaijaPrimeSchool.Infrastructure/DependencyInjection.cs	Registered the 3 new services.
src/NaijaPrimeSchool.Infrastructure/Persistence/ApplicationDbContext.cs	Added 8 DbSets, ConfigureFamily.
src/NaijaPrimeSchool.Infrastructure/Persistence/DatabaseInitializer.cs	Seeded Relationships, EnrolmentStatuses, BloodGroups, MaritalStatuses.
src/NaijaPrimeSchool.Infrastructure/Services/LookupService.cs	Added 7 new lookup methods.
Web layer (new)	—
src/NaijaPrimeSchool.Web/Components/Pages/Family/Students.razor	Students directory + filters.
src/NaijaPrimeSchool.Web/Components/Pages/Family/CreateStudent.razor	Create student form.
src/NaijaPrimeSchool.Web/Components/Pages/Family/EditStudent.razor	Profile / Parents / Enrolment tabs.
src/NaijaPrimeSchool.Web/Components/Pages/Family/Parents.razor	Parents directory + filters.
src/NaijaPrimeSchool.Web/Components/Pages/Family/CreateParent.razor	Create parent form.
src/NaijaPrimeSchool.Web/Components/Pages/Family/EditParent.razor	Profile / Linked pupils tabs.
src/NaijaPrimeSchool.Web/Components/Pages/Family/Enrolments.razor	Enrolment overview + filters.
Web layer (modified)	—
src/NaijaPrimeSchool.Web/Components/_Imports.razor	Added Family + Family.Dtos usings.
src/NaijaPrimeSchool.Web/Components/Layout/NavMenu.razor	Added the Family panel.
Tooling (new)	—
tools/generate_sprint3_guide.py	This document's generator.

— End of the Sprint 3 implementation guide. The next sprint lands attendance on top of the (Student × Class) primitives established here.